Reliability Engineering and System Safety 94 (2009) 838–854

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

Modelling and optimization of proof testing policies for safety instrumented systems

A.C. Torres-Echeverría [a,*], S. Martorell [b], H.A. Thompson [a]

[a] Department of Automatic Control and System Engineering, The University of Sheffield, Mappin Street, Sheffield S1 3JD, UK
[b] Department of Chemical and Nuclear Engineering, Polytechnic University of Valencia, Camí de Vera sn, 4602, Valencia, Spain

ARTICLE INFO

Article history:
Received 16 May 2008
Received in revised form
25 August 2008
Accepted 13 September 2008
Available online 8 October 2008

Keywords:
Safety system
Proof test
Test strategy
Probability of failure on demand
Spurious trip rate
Common cause failure
Diagnostic coverage
IEC 61508
Optimization
Genetic algorithms

ABSTRACT

This paper introduces a new development for modelling the time-dependent probability of failure on demand of parallel architectures, and illustrates its application to multi-objective optimization of proof testing policies for safety instrumented systems. The model is based on the mean test cycle, which includes the different evaluation intervals that a module goes periodically through its time in service: test, repair and time between tests. The model is aimed at evaluating explicitly the effects of different test frequencies and strategies (i.e. simultaneous, sequential and staggered). It includes quantification of both detected and undetected failures, and puts special emphasis on the quantification of the contribution of the common cause failure to the system probability of failure on demand as an additional component. Subsequently, the paper presents the multi-objective optimization of proof testing policies with genetic algorithms, using this model for quantification of average probability of failure on demand as one of the objectives. The other two objectives are the system spurious trip rate and lifecycle cost. This permits balancing of the most important aspects of safety system implementation. The approach addresses the requirements of the standard IEC 61508. The overall methodology is illustrated through a practical application case of a protective system against high temperature and pressure of a chemical reactor.

© 2008 Elsevier Ltd. All rights reserved.

1. Introduction

Proof testing is a periodic activity that mainly verifies that the specified safety integrity level of the safety system is met and kept during the system lifecycle. Its more direct objective is to detect dangerous unrevealed failures. It has therefore a fundamental importance for safety instrumented systems (SIS). As a consequence, periodic proof testing can contribute to achieving and improving the safety integrity level of the system without making modifications to the safety system design. However, testing can also convey some collateral adverse effects since it is not a totally innocuous activity. It is therefore necessary to find an optimal planning of the testing activity throughout the overall operational life of the system, so that a good balance between its benefits and its direct and indirect costs is achieved.

This paper has a double objective: Firstly, it aims to introduce a new model for quantification of time-dependent probability of failure on demand (PFD) of SIS. Secondly, it presents the integration of this PFD model into the multi-objective optimization of proof testing policies of SIS based on genetic algorithms (GA). The article also presents a deep review of proof testing optimization and the current standardization and practice for SIS, specifically in the process industry. With this, the modelling and optimization is tailored for a practical approach.

The PFD is the safety unavailability of the system [1]. It is the fraction of time that the system is unavailable to perform its protective function when the plant is operating. Therefore, it can be modelled using classical methods such as Fault Tree Analysis, Markov Analysis, Reliability Block Diagrams, etc. Based on these methods, several models have been developed in order to quantify the average PFD (PFD_{avg}), notably IEC 61508 [2], ISA TR84.02 [3], Goble [1] and the PDS method [4]. Perhaps the only method that has the flexibility to include quantification for the changing test strategy could be Markov Analysis, which is capable of accommodating failure and repair processes. However, it is well known that its complexity grows exponentially with the number of the system nodes and states.

The time-dependent PFD algorithm has been developed with the specific aim to evaluate the impact that different test frequencies and strategies have on the PFD_{avg}. This is the measure of safety integrity for SIS. It also intends to include explicitly the common cause failure rate of the system components (using the β-factor model) and their diagnostic coverage. It is primarily based on the previous work developed by Martorell et al. [5,6], Cepin and Mavko [7] and Cepin [8], which was primarily intended for surveillance requirements of nuclear power plants (NPP). The model addresses the level of detail required by IEC 61508 [2] for the quantification of

* Corresponding author. Tel.: +44 7951231263; fax: +44 1142225138.
E-mail address: a.torres@sheffield.ac.uk (A.C. Torres-Echeverría).

0951-8320/$ - see front matter © 2008 Elsevier Ltd. All rights reserved.
doi:10.1016/j.ress.2008.09.006

Nomenclature

CCF	common cause failure
FC	final control subsystem (valve and actuator)
GA	genetic algorithm
HPIS	high-pressure injection system
IEC	International Electrotechnical Commission
LCC	lifecycle cost
LS	logic solver subsystem
MO	multi-objective
MooN	M-out-of-N voting system
NPP	nuclear power plant
NSGA-II	non-dominated sorting GA
PFD	probability of failure on demand
PLC	programmable logic controller
PT	pressure measurement/transmitter subsystem
RAMS+C	reliability, availability, maintainability and safety+-cost
SIL	safety integrity level
SIS	safety instrumented system
SO	single objective
STR	spurious trip rate
TI	test interval
TS	test strategy
TT	temperature measurement/transmitter subsystem
ρ_i	remaining slots
C_{ACC}	cost of accident
C_{CM}, C_i^{CM}	corrective maintenance cost, cost per event
C_{HAZARD}	cost per accident event
C_{hr}	repair hourly cost
C_{ht}	test hourly cost
C_i	cost item for ith component
C_{MooN}	beta factor modifier for MooN architecture
$C^{purchase}$	acquisition cost
C_{RISK}	risk cost
C_{SD}	cost per spurious shutdown
C_{SP}	average cost of expenditure of spares per repair
C_{STR}	cost of spurious trip rate
$C_{T\&CM}$	cost of test and corrective maintenance
C_T, C^T_i	proof test cost, proof test cost per event
$F(ACC\|PFD_{avg})$	accident frequency without SIS per year
f_i	Front number ith
f_j^{CM}	repair frequency of i component
i	ith component being tested or last tested subscript
I_i	importance measure for cut set i
K	staggering factor
$LCC_{T\&CM}$	test and corrective maintenance related LCC
mod	modulo operator
n	number of point evaluations
N, N_i	number of components, number of ith components
n_i	number of individuals in each front
N_{pop}	size of population
N_{tests}	number of tests
N_{trips}	number of spurious trips
P	multiplication factor
$\boldsymbol{P}$	array of the initial parent population
$P(C_i)$	probability of cut set i occurring
$PFD(t)$	time-dependent PFD
$PFD(t_i)$	time-dependent PFD point evaluation
PFD_{avg}	average probability of failure on demand
$PFD_{CCF}(t)$	time-dependent PFD from CCF
PFD_{IND}	total independent PFD
$PFD_{ind\ i}(t)$	PFD(t) from independent failure of the ith component
PFD_{max}	maximum (peak) PFD(t)
$PFD_{TOT}(t)$	total time-dependent PFD
P_{r-trip}	probability of test-induced spurious trip
PVF	factor by present value
Q	array of the offspring population
q	constant unavailability symbol
$Q_{SYS}(q(t))$	probability of system unavailability
R	array of the initial+offspring populations
r	reduction rate parameter for NSGA-II
SD_{LOSS}	cost of production loss per hour
SD_{Time}	restart time after shutdown
STR_λ	STR caused by internal failure
STR_T	total spurious trip rate
STR_{test}	proof test-induced STR
T	total evaluation time (mission time)
t	time
T_P, T_{Pi}	time to first test, T_P of the ith component
T_{P1}	time to first test of the first component
T_{PN}	time to first test of the Nth component
T_r	repair time
T_t	test time
w	test interval remainder parameter
w_{CCF}	test interval remainder parameter for CCF
w_i	test interval remainder of the ith component
$\boldsymbol{x}$	decision variables vector
$\boldsymbol{Y}$, $\boldsymbol{F}(\boldsymbol{x})$	objective function vector
β	beta factor
β', β_{MooN}	beta factor modified for MooN architecture
β^D	beta factor for detected failures
β_{N-1}	beta factor for $N-1$ components
β^U	beta factor for undetected failures
ε	diagnostic coverage
ε^{Danger}	diagnostic coverage for dangerous failures
ε^{Safe}	diagnostic coverage for safe failures

Failure rates:

λ	failure rate
λ^T	total failure rate
λ^{Danger}	dangerous failure rate
λ^{Safe}	safe failure rate
λ^D	detected
λ^U	undetected
λ^{DN}	detected normal (random)
λ^{DC}	detected common cause
λ^{UN}	undetected normal
λ^{UC}	undetected common cause
λ_{N-1}^{DC}	detected common cause for $N-1$ components
λ_{N-1}^{UC}	undetected common cause for $N-1$ components

PFD_{avg}. The model aims to be able to accommodate changing conditions on the test strategy of the system components, with the objective of being suitable for multi-objective optimization problems. Therefore, it requires a treatment that at the same time includes the modelling detail required for a real SIS application, and it does not present the disadvantage of excessive complexity growth.

2. SIS requirements and proof testing practices

IEC 61511 [9], a standard specific for the process industry, defines a SIS as an "instrumented system used to implement one or more safety instrumented control functions. A SIS is composed of any combination of sensors, logic solver and final elements".

IEC 61508 is a generic standard for functional safety of electrical, electronic and programmable electronic safety systems, and its requirements are applicable to SIS. The main requirement of the standard is that every safety function executed by the system must achieve a determined safety integrity level (SIL). This is a quantitative index indicating the acceptable probability of failure that a system can have to consider it appropriate for a given specific safety integrity requirement. For low-demand operating systems, IEC 61508 defines the SIL levels in terms of the *average probability of failure* [of the safety system] *to perform its design function on demand* [because it is unavailable]. This is abridged as the average probability of failure on demand (PFD_{avg}), which is also the term used by IEC 61511. Thus, the PFD_{avg} is in fact the unavailability of the system that affects its ability to react to hazards; i.e. the safety unavailability [1], and it is related to unreliability and downtimes attributable only to dangerous failures (no safe failures). The definition of SIL levels can be seen in Table 1.

It is worth making a clarification about different system failure modes. SIS can fail in two different ways: Dangerous and safe. IEC 6108 defines dangerous failures as those "which has the potential to put the safety-related system in a hazardous or fail-to-function state". In other words, they are those failures that prevent the system to perform its intended function on a real demand to do so. In contrast, safe failures are those which do not threat the ability of the safety system to perform its intended function. However, they can lead the system to operate without an actual demand; i.e. spurious operation. Thus, it can be concluded that the dangerous failures contribute to the system PFD_{avg}, while, in contrast, the safe failures contribute to the system spurious trip rate (STR).

The definition of proof test given by IEC 61508 [2] is a "periodic test performed to detect failures in a safety-related systems so that the system can be restored to an "as new" condition or as close as practical to this condition". The need of routine maintenance action in order to detect unrevealed failures is clearly established by the standard, and proof test is one of these activities. Therefore, it has an important role in the achievement of safety integrity. According to IEC 61508-2, the frequency of proof test will be dependent upon the target failure measure associated with the SIL, the architecture, the automatic diagnostic coverage and the expected demand rate. IEC 61508-6 provides tables to determine PFD_{avg} for subsystems tested specifically at 6 months, 1, 2 and 10 years intervals. No more specific guidance is provided. Notice that IEC 61508-7 defines "functional testing" as an activity "to reveal failures during the specification and design phases." Therefore, proof and functional testing are not the same. IEC 61511 [9] states that the frequency of proof test "shall be decided using the PFD_{avg} calculation". It mentions that the different parts of the SIS may require different test frequencies.

The Norwegian Oil Industry Association [10] requires the proof test to confirm correct operation for the entire SIS loop, including sensors, logic solver and final control elements. It provides a list of aspects to be included in the proof tests, summarized in complete system functionality. The document establishes that the test should preferably test the entire safety loop at once (integral test). This same requirement was laid out by ISA 84.01-1996 [11] (the first standard to formally address SIS specifically for the process industry), as it requires the final element actuation to be tested in response to sensor inputs. However, if this is not possible, partial test (by subsystem) is accepted under certain conditions. This standard considered the periodically scheduled functional testing as part of the SIS regular maintenance. It established the necessity of providing test facilities as an integral part of the SIS. According to ISA 84-01-1996, the functional test interval (TI) should be selected to achieve the system SIL.

American Petroleum Institute's API RP-14C [12], is the more prescriptive standard. It establishes the necessity of carrying out *performance testing* for confirmation of the ability of the systems *to perform the design safety function*. It establishes as a requirement the execution of the operational test at least once per year. However, it permits use of an alternative larger TI as long as it ensures the same or higher reliability. Some guidance is given, advising monthly test of pneumatic sensors, and quarterly for electronic transmitters. This standard indicates intervals between 3 and 24 months and up to 36 or 60 for some non-critical equipment.

It can be noticed that there is no a generic thoroughly developed clear guidance about the test frequency, scope, and procedures for the process industry (comparing with the nuclear industry, e.g. NUREG/CR-6141 [13]). This lack of standardization was implicitly recognized by the British Health and Safety Executive (HSE), when commissioning a study about actual proof testing practices in the chemical industry [14]. The study confirmed the "existence of conflict between the need for realistic proof testing and the need to minimise downtime" (i.e. system unavailability). It establishes that the purpose of the proof testing is the detection of unrevealed fail-to-danger faults at the time of testing (different from automatic diagnostics that intend to reveal faults between tests). It also requires including all the subsystems (sensors, logic solvers and final elements) in the proof test. Although the study describes the end-to-end testing as the ideal practice, it acknowledges the need to resort to partial testing as a necessary practice under certain conditions, since end-to-end testing is not always practicable. This document gives a definition for partial testing, which is considered either "the testing of system components at different times and frequencies or the testing of sub-sets of functions of single components". As other documents, the guidance provided about the frequency of tests is quite loose.

In summary, there exists a trend to use the terms "functional test" and "proof test" as the same thing. However, they are not completely the same. We consider the IEC 61508 definition of proof test as the more convenient, and thus it is the one we use in this work. The objective of proof test is to detect unrevealed failures (especially dangerous ones), and it should include both inherent and functional integrity. Some standards require considering the inclusion of the necessary facilities so online testing is possible if required [11]. Proof testing is required to confirm correct operation for the entire SIS loop, including sensors, logic solver and final control elements. It should be an integral end-to-end test [10,11]. However, when this is not practicable partial test is widely accepted [11,14]. Proof test frequency should be specified according to the required system safety integrity and its design [2,9,11]. There is no more specific guidance or constraints about how to set the test frequency. Only few numerical examples were found [2,12]. It can be concluded that during optimization of test frequency and strategy, partial test seems to be the most convenient for manipulation rather than integral test. This is feasible since redundancies in the SIS are usually at the subsystem level rather than at the system level.

Table 1
SIL for low-demand mode of operation (IEC 61508-1).

SIL	PFD_{avg}
4	$\geqslant 10^{-5}$ to $<10^{-4}$
3	$\geqslant 10^{-4}$ to $<10^{-3}$
2	$\geqslant 10^{-3}$ to $<10^{-2}$
1	$\geqslant 10^{-2}$ to $<10^{-1}$

3. Overview of testing modelling and optimization

An early analysis for finding the optimal TI of safety-related equipment of NPPs was made by Martorell et al. [5]. It established the average test cycle and quantified unavailability as the contribution of failure between tests, test time and repair time, obtaining simplified analytical equations for the sequential and staggered test strategies of a two-component parallel system. Subsequently, Uryas'ev and Vallerga [15] proposed a general approach for optimization of TIs of standby safety systems. They identified four states of the instantaneous unavailability of tested components. The unavailability was estimated using continuous time Markov models, and cut set quantification for calculation of the time-dependent unavailability. Schofield [16] approached the optimization of proof TIs using fault tree analysis. Vaurio [17] developed analytical expressions for quantification of average unavailability for sequential and staggered testing (uniformly distributed over the test interval time). In a subsequent work [18], he carried out a study for optimization of test and maintenance intervals at component level (for series systems) and plant level (using fault trees' minimal cut stets). Martorell et al. [6] approached the surveillance TI requirements identifying the unavailability components of the periodic test cycle and developed analytical expressions based on probabilistic methods. The concept of critical safety unavailability was addressed by Hokstad et al. [19], who developed a model for its quantification, which accounts for both manual and automatic (diagnostics) tests, failures introduced during testing and the probability of imperfect test, and applied it to single fire and gas detectors.

Cepin and Mavko [7] established an unavailability time-dependent probabilistic model adaptable to several test strategies. The model was later applied in Cepin [8] for optimization of safety equipment outages scheduling due to testing and maintenance by simulated annealing. Bukowski [20] presented a development that permitted inclusion of periodic inspections and repairs into Markov models of safety-critical systems. Vaurio [21] developed equations for including common cause failures in unavailability fault trees, utilizing both implicit and explicit methods for standby safety systems, and addressed the effects of test intervals and test staggering. Lapa et al. [22] presented a primer approach to include non-periodic tests (within an established period) adapted to optimization seasonal constrained problems. The problem studied considered a single component. Curtois and Delsarte [23] obtained an analytic closed form expression of the optimum inspection interval for m of n ($m<n$) redundant components (perfect inspection). Staggered inspection was considered as well. The function aims to minimize the probability of the system becoming unavailable (failure) between inspections, and the frequency of inspections.

As mentioned above, the testing activity also conveys some adverse effects. Obviously, the most direct is the incurred test down time. Nevertheless, other adverse effects may be also significant, one of the most relevant being the human error induced spurious trips. Kim et al. [24,25] made a thorough analysis of test adverse effects over plant risk (availability). Vaurio [18] contemplated the failure modes induced by human error in the optimization of test and maintenance intervals. Hokstad et al. [19] accounted failures induced during test and the probability of imperfect test in reliability models. Bukowski [20] included imperfect inspection and repair into Markov models of safety-critical systems. Zhao et al. [26] approached the evaluation and optimization of reliability at component level, considering imperfect inspections at non-constant intervals. Lundteigen and Rausand [27] presented the first monothematic study about system spurious activation of SIS, where they discussed the increment of spurious activations by human error during test.

Optimization of testing has been addressed under diverse approaches. One evolving and actually researched area is optimization with GA. Pattison and Andrews [28] included maintenance test intervals as one of several decision variables for safety system design optimization, solving it by a single objective (SO) GA, incorporating the use of binary decision diagrams for quantification of the dependability objective function.

Martorell et al. [29] refined and applied the unavailability model developed in [25] for optimization of TIs for the high-pressure injection system (HPIS) of a NPP using a SO GA. The optimization was made considering risk (unavailability) and cost. Marseguerra and Zio [30] implemented a GA in combination with Monte Carlo simulation for optimization of maintenance and repair policies. Busacca et al. [31] introduced the application of multi-objective (MO) GAs to safety systems, with an application to optimization of TIs for a HPIS, based on mean availability, cost and risk (exposure time). The cost function included surveillance and maintenance plus the cost of an accident (plant damage). Martorell et al. [32] combined the optimization of TIs, preventive maintenance and allowed outage times of a NPP, so the technical specifications and maintenance requirements were addressed integrally.

Andrews and Bartlett [33] applied Ref. [28] methodology to a larger problem: a water deluge system, under maintenance and spurious trip rate costs constraints. Vinod et al. [34] applied a GA to optimization of in-service inspection (which included functional test).

Martorell et al. [35] reformulated its approach as a general multi-objective RAMS+C optimization, and refined it in [36]. This formalized the method for assisting the decision-making process in NPPs with a strategy based on: (1) technical specifications and maintenance, (2) RAMS plus cost, and (3) goals included as constraints.

Martorell et al. [37] explored the multi-objective optimization of the plant surveillance requirements simultaneously optimizing the TIs and the test planning (strategy), based on time-dependent modelling. Martorell et al. [38] treated the HPIS system optimization with a double-nested loop. An external loop optimizes the TIs with a learning machine, while the internal loop (a MO GA) optimizes the test staggering.

Torres-Echeverria and Thompson [39] approached the inclusion of system safety integrity requirements specified by IEC 61508 in multi-objective optimization of system design, and applied it using a genetic algorithm. In a subsequent work, Torres-Echeverria et al. [40] analyzed the inclusion of diversity as a measure for counteracting common cause failure (CCF) for redundancy allocation in multi-objective design optimization of parallel-redundant safety systems.

4. Testing basic concepts

The proof test requirements of a SIS are addressed by the test policy. A proof testing policy includes the type of test, the test interval (TI, which determines the frequency), and the test strategy (TS). These two last items, denominated together TI+TS, can be manipulated to find an optimal solution [37,38]. They are defined under the framework of the mean test cycle, which is described below.

4.1. Mean test cycle

Proof testing is a periodically executed activity. The mean test cycle includes all the events between two consecutive tests of a component [6]. The component goes through several states along the cycle: testing, repair and standby (with respect to test) or time

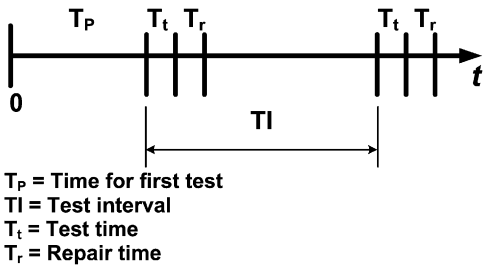


Fig. 1. Mean test cycle.

between tests. There is additionally a previous time before the first test of the component which is necessary to consider for quantification of unavailability. Fig. 1 illustrates the description of the mean test cycle. Each of the states described in this figure has a contribution to the component unavailability. T_P is the time that elapses between the first system start up to the time of first test of the component. The test duration time T_t and the repair time T_r are assumed to be mean values. The time between two consecutives tests of the same component is equivalent to $TI-T_t-T_r$.

4.2. Test strategies

The TS establishes how the tests of the redundant components are scheduled with respect to one another. Proof testing can be implemented using several different strategies. A general classification of TS is listed below, where T_{Pi} is the time of first test of the ith component, T_t the duration time of the test, and TI the test interval.

- *Simultaneous test*: The N redundant components are tested at the same time, i.e. $T_{P1} = T_{P2} = \cdots = T_{PN}$. This implies that there are N crews available to test the N components. This is a strategy not suitable for safety systems that must remain in service permanently, since the system is made unavailable during the test.
- *Sequential test*: The N redundant components are tested consecutively one after another. Immediately after one component is tested and restored to work the next one is tested and so on until finishing with all the components of the subsystem. For simplification, it can be said that the only difference between the test events of two components is the time the component is under test T_t, i.e. $T_{P2} = T_{P1}+T_t \cdots T_{PN} = T_{PN-1}+T_t$. The components are usually tested at the beginning or end of the test interval TI [41].
- *Staggered test*: The N redundant components are tested with a difference of time of TI/N. This is the most common staggering. This article explores, however, a staggered strategy where the components are scheduled for a test with difference in time larger than T_t, but not necessarily informingly distributed as TI/N. We have called the former non-uniform staggered test, and the latter uniform staggered test.
- *Independent test*: The relative test time of the N components are tested without any specific schedule, with a random difference of time between one another.

5. PFD time-dependent model

5.1. PFD(t) basic model

The PFD is modelled as unavailability given that it is the system safety unavailability. The model has been developed using as departure point the model presented by Ref. [7]. Consider the following definition:

$$w = (t - T_P) mod\ TI \tag{1}$$

where mod is the modulo operation (dividing remainder), which permits to reset w every time a test interval is completed and a new test must be performed.

The contributions that each stage of the mean test cycle has to the unavailability are formulated below:

(a) *Standby before first test*: The unavailability is simply the temporal contribution due to the component unreliability plus a constant q. This parameter represents a constant that depending of the application can include different contributions [18], for instance the unavailability caused by detected failures (which will be explored later).

$$PFD(t) = q + 1 - e^{-\lambda t} \quad \text{for } t - T_P \leqslant 0 \tag{2}$$

(b) *Testing*: The component is taken out of service so it can be tested. It is assumed that it is unavailable during the entire test time.

$$PFD(t) = 1 \quad \text{for } \{t - T_P > 0\ \&\ 0 < w \leqslant T_t\} \tag{3}$$

(c) *Repair*: This contribution to the PFD is due to the possibility that the component is found failed during test, and thus it must be repaired (assuming it is repaired immediately after test). It is clear that the unavailability is composed of two terms: one where the component is failed (thus under repair) subject to the probability of failure, the other where the component is not failed, i.e. working, subject to the probability of no failure. Notice the difference between the formulation after the first test and the subsequent tests.

$$PFD(t) = \begin{cases} q + 1 - e^{-\lambda T_P} + (e^{-\lambda T_P} - q)(q + 1 - e^{-\lambda(w-T_t)}) \\ \quad \text{for } \{0 < t - T_P \leqslant TI\ \&\ T_t < w < T_t + T_r\} \\ q + 1 - e^{-\lambda(TI-T_t-T_r)} + (e^{-\lambda(TI-T_t-T_r)} - q)(q + 1 - e^{-\lambda(w-T_t)}) \\ \quad \text{for } \{t - T_P > TI\ \&\ T_t < w < T_t + T_r\} \end{cases} \tag{4}$$

(d) *Standby between tests*: The quantification is the same as the first standby interval, just different in the model's exponents.

$$PFD(t) = q + 1 - e^{-\lambda(w-T_t-T_r)} \quad \text{for } \{t - T_P > 0\ \&\ w > T_t + T_r\} \tag{5}$$

It is important to note that the PFD(t) goes through an initial (transient) state before becoming periodically steady. For convenience of formulation, it can be considered that the initial state comprises the time from $t = 0$ to T_P+TI. Later it will be seen that this time will be considered as $t = T_{P1}$+TI for redundant systems.

5.2. Inclusion of the effect of automatic diagnostics

Components for safety systems are usually empowered with automatic diagnostics, which have an important effect on the reduction of their PFD. Their effectiveness is measured as the percentage of the total failure rate that is detected, called the diagnostic coverage (ε). As a result, the provision of an automatic in-built diagnostics mechanism splits the total failure rate into two failure modes: Detected and undetected.

$$\lambda^T = \varepsilon\lambda^T + (1 - \varepsilon)\lambda^T = \lambda^D + \lambda^U \tag{6}$$

According to Ref. [18], the parameter q in the basic model (Eqs. (2)–(5)) can represent different contributions to the unavailability attributed to detected failures. One of these possibilities is the unavailability attributed to detected failures, assuming that they

are detected immediately (i.e. negligible failure residence time). Therefore, it is possible to reformulate the PFD(t) basic model in terms of detected and undetected failures, where the former are modelled as a constant, and the contribution by undetected failures shapes the time dependency of the PFD. Therefore, the constant q of the basic model is substituted by the detected failures share of Eq. (6) in the subsequent formulations.

5.3. The contribution of independent failures

It is known that common cause failure (CCF) is the phenomenon where two or more redundant components fail due to the same cause. The β-factor model is the most widely used for quantification of CCF [42]. According to this model, the total failure rate is split into independent and common cause failures. The contribution of CCF to the PFD(t) is quite complex to model. Therefore, the contribution of independent failures to the PFD(t) is modelled first. The contribution of CCF is modelled in Section 5.4.

We are applying the C_{MooN} modification factor in order to modify the basic β-factor according to the system voting logic (refer to [4] for a review of the origin of the modification factor).

$$\beta' = \beta_{\mathrm{MooN}} = \beta\, C_{\mathrm{MooN}} \tag{7}$$

For parallel systems $M = 1$. Thus, the modification factor becomes $C_{1\mathrm{ooN}}$. Applying this concept to Eq. (6), the detected and undetected failure modes are divided into independent and common cause failures, and thus the total failure rate is split into four factors

$$\lambda^{\mathrm{T}} = (1-\beta'^{\mathrm{D}})\varepsilon\,\lambda^{\mathrm{T}} + \beta'^{\mathrm{D}}\,\varepsilon\,\lambda^{\mathrm{T}} + (1-\beta'^{\mathrm{U}})(1-\varepsilon)\lambda^{\mathrm{T}} + \beta'^{\mathrm{U}}(1-\varepsilon)\lambda^{\mathrm{T}} \tag{8}$$

The four failure modes can then be renamed to simplify the formulation

$$\lambda^{\mathrm{T}} = \lambda^{\mathrm{DN}} + \lambda^{\mathrm{DC}} + \lambda^{\mathrm{UN}} + \lambda^{\mathrm{UC}} \tag{9}$$

Eq. (8) indicates a difference between β-factors for detected and undetected failures (IEC 61508-6). Nevertheless, some methods [4] do not distinguish this difference. We leave to the reader's discretion the application of the most relevant value for β.

The model of the PFD(t) due to the contribution of independent failures, $\mathrm{PFD}_{\mathrm{ind}}(t)$, is derived by simply substituting Eq. (9) into Eqs. (2)–(5). In order to present the complete model at single glance, all its formulas have been put together in one single page. The contribution of independent failures is given by Eqs. (12)–(15). Notice that this set of equations represents the $\mathrm{PFD}_{\mathrm{ind}\ i}(t)$ for each single component, being the index i the correspondence to the ith component.

5.4. The contribution of common cause failure

CCF is usually modelled in probabilistic analysis as an additional component in series with the redundant system. A fault tree depicting this situation for a parallel system of three components is shown in Fig. 2. It is then convenient to create an individual model for the CCF component. Fig. 3 shows the $\mathrm{PFD}_{\mathrm{ind}}(t)$ for all the system components, with a hypothetical plot of the PFD(t) due to the CCF component, denominated $\mathrm{PFD}_{\mathrm{CCF}}(t)$.

Several remarks can be made observing Fig. 3:

- The CCF unavailability follows the same structure of the basic model for the mean test cycle (Fig. 1): A preliminary interval before first test, followed by test, repair and standby states.
- The mean test cycle for the CCF is repeated N times (being N the number of the redundant components) during the TI. The mean test cycle is reinitiated every time a component is tested.
- It is clear that after some time the mean test cycle of the CCF will repeat itself periodically, which for formulation convenience has been placed after the time of the first test of the first component plus the test interval (T_{P1}+TI). Therefore, as shown in Fig. 3, we can say that the time previous to T_{P1}+TI corresponds to the initial state of the $\mathrm{PFD}_{\mathrm{CCF}}(t)$, and the time afterwards corresponds to its steady state.

Following these remarks, the model has been derived for the $\mathrm{PFD}_{\mathrm{CCF}}(t)$. After modelling unavailability by CCF for two, three and four components in parallel, it was observable that there existed a pattern, which is repeated for N number of components in parallel (larger than two). This was the basis for making a formulation of the $\mathrm{PFD}_{\mathrm{CCF}}(t)$ model for N number of components. The model would be the same for each component as the basic model (including detected and undetected failures), but the time for first test per component $T_{\mathrm{P}i}$ and the parameter w_i would change. For the model of $\mathrm{PFD}_{\mathrm{ind}\ i}(t)$ we denominate for the ith component

$$w_i = (t - T_{\mathrm{P}i}) mod\ \mathrm{TI} \tag{10}$$

Since the $\mathrm{PFD}_{\mathrm{CCF}}(t)$ periodicity depends on the moment of the first test of the first component T_{P1} plus the test interval TI, the parameter w for the CCF becomes equivalent to that for the first component being tested

$$w_{\mathrm{CCF}} = w_1 = (t - T_{\mathrm{P1}}) mod\ \mathrm{TI} \tag{11}$$

With this concept it is possible to formulate the contribution of CCF to the $\mathrm{PFD}_{\mathrm{CCF}}(t)$, presented in Eqs. (16)–(19). Notice that the formulation for standby before first test, repair and standby between tests (Eqs. (16), (18) and (19)) is basically the same as in the basic model, just reformulated (from Eqs. (2)–(5)) to accommodate the CCF contribution. Nevertheless, the $\mathrm{PFD}_{\mathrm{CCF}}(t)$ during test time (Eq. (17)) required a different formulation considering several factors. First of all consider that during test

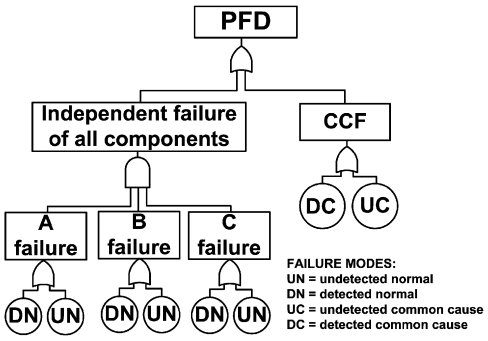


Fig. 2. Fault tree for a three-component parallel system.

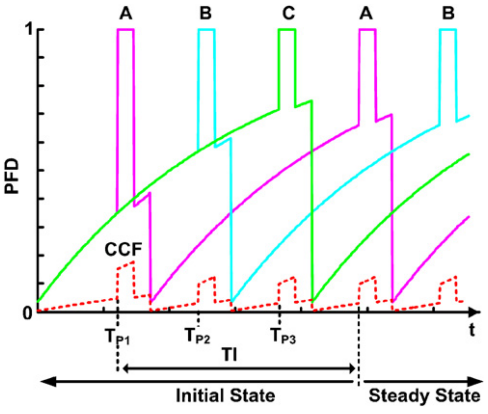


Fig. 3. PFD(t) for the three-component system with CCF.

the $\mathrm{PFD}_{\mathrm{CCF}}(t)$ requires a formulation that takes into account that one of the components is unavailable. This means that the CCF is only due to $N-1$ components. Therefore, the β-factor is modified during the duration of the test. For a system with two components, the CCF becomes zero ($\beta = 0$), and for a system with $N>2$, the CCF becomes equivalent to the contribution of $N-1$ components (β_{N-1}). In addition, if the difference in time between tests of the components is less than T_{t}, this implies that it is a simultaneous TS. Thus, no CCF can occur since all the components are unavailable. Therefore, for cases in which $T_{\mathrm{P}i+1}-T_{\mathrm{P}i}<T_{\mathrm{t}}$, the $\mathrm{PFD}_{\mathrm{CCF}}(t)$ is zero. The complete $\mathrm{PFD}(t)$ model is contained in Eqs. (12)–(19). It is important to remark again that in each equation the subscript i stands for the ith component being tested or last tested.

PFD(t) complete model
Independent failures contribution

Standby before first test : (12)

$$\mathrm{PFD}_{\mathrm{ind}\ i}(t) = \lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}} t} \quad \text{for } t - T_{\mathrm{P}i} < 0$$

Testing : (13)

$$\mathrm{PFD}_{\mathrm{ind}\ i}(t) = 1 \quad \text{for} \quad \{t - T_{\mathrm{P}i} \geqslant 0\ \&\ 0 \leqslant w_i < T_{\mathrm{t}}\}$$

Repair : (14)

$$\mathrm{PFD}_{\mathrm{ind}\ i}(t) = \begin{cases} \lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}} T_{\mathrm{P}i}} + (\mathrm{e}^{-\lambda^{\mathrm{UN}} T_{\mathrm{P}i}} - \lambda^{\mathrm{DN}} T_{\mathrm{r}})(\lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}}(w_i - T_{\mathrm{t}})}) \\ \quad \text{for } \{0 \leqslant t - T_{\mathrm{P}i} < \mathrm{TI}\ \&\ T_{\mathrm{t}} \leqslant w_i < T_{\mathrm{t}} + T_{\mathrm{r}}\} \\ \lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}}(\mathrm{TI} - T_{\mathrm{t}} - T_{\mathrm{r}})} + (\mathrm{e}^{-\lambda^{\mathrm{UN}}(\mathrm{TI} - T_{\mathrm{t}} - T_{\mathrm{r}})} - \lambda^{\mathrm{DN}} T_{\mathrm{r}})(\lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}}(w_i - T_{\mathrm{t}})}) \\ \quad \text{for } \{t - T_{\mathrm{P}i} \geqslant \mathrm{TI}\ \&\ T_t \leqslant w_i < T_{\mathrm{t}} + T_{\mathrm{r}}\} \end{cases}$$

Standby between tests : (15)

$$\mathrm{PFD}_{\mathrm{ind}\ i}(t) = \lambda^{\mathrm{DN}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UN}}(w_i - T_{\mathrm{t}} - T_{\mathrm{r}})}$$
$$\text{for } \{t - T_{\mathrm{P}i} \geqslant 0\ \&\ w_i \geqslant T_{\mathrm{t}} + T_{\mathrm{r}}\}$$

Common cause failures contribution

Standby before first test : (16)

$$\mathrm{PFD}_{\mathrm{CCF}}(t) = \lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}} \cdot t} \quad \text{for } t - T_{\mathrm{P1}} < 0$$

Testing : (17)

IF $T_{\mathrm{P}i+1} - T_{\mathrm{P}i} < T_{\mathrm{t}}$ (Simultaneous test)

$$\mathrm{PFD}_{\mathrm{CCF}}(t) = 0 \quad \text{for} \left\{ t - T_{\mathrm{P1}} \geqslant 0\ \& \left| \begin{array}{l} 0 \leqslant w_1 < T_{\mathrm{t}} \\ T_{\mathrm{P}i} - T_{\mathrm{P1}} \leqslant w_1 < T_{\mathrm{P}i} - T_{\mathrm{P1}} + T_{\mathrm{t}} \end{array} \right.$$

IF $T_{\mathrm{P}i+1} - T_{\mathrm{P}i} \geqslant T_{\mathrm{t}}$ (Non-simultaneous test)

$$\mathrm{PFD}_{\mathrm{CCF}}(t) = \begin{cases} \lambda^{\mathrm{DC}}_{N-1} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}_{N-1} t} \quad \text{for } \{0 \leqslant t - T_{\mathrm{P1}} < \mathrm{TI}\ \&\ 0 \leqslant w_1 < T_{\mathrm{t}}\} \\ \lambda^{\mathrm{DC}}_{N-1} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}_{N-1}(w_1 - (T_{\mathrm{P}i-1} - T_{\mathrm{P1}}) - T_{\mathrm{t}} - T_{\mathrm{r}})} \quad \text{for } \{0 \leqslant t - T_{\mathrm{P1}}\ \&\ T_{\mathrm{P}i} - T_{\mathrm{P1}} \leqslant w_1 < T_{\mathrm{P}i} - T_{\mathrm{P1}} + T_{\mathrm{t}}\} \\ \lambda^{\mathrm{DC}}_{N-1} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}_{N-1}(w_1 + \mathrm{TI} - (T_{\mathrm{PN}} - T_{\mathrm{P1}}) - T_{\mathrm{t}} - T_{\mathrm{r}})} \quad \text{for} \{t - T_{\mathrm{P1}} \geqslant \mathrm{TI}\ \&\ 0 \leqslant w_1 < T_{\mathrm{t}}\} \end{cases}$$

where

$$\lambda^{\mathrm{DC}}_{N-1} = \beta'^{\mathrm{D}}_{N-1} \varepsilon \lambda^{\mathrm{T}} = \beta^{\mathrm{D}}\, C_{1\mathrm{oo}(N-1)}\, \varepsilon \lambda^{\mathrm{T}}$$
$$\lambda^{\mathrm{UC}}_{N-1} = \beta'^{\mathrm{U}}_{N-1} (1-\varepsilon) \lambda^{\mathrm{T}} = \beta^{\mathrm{U}}\, C_{1\mathrm{oo}(N-1)} (1-\varepsilon) \lambda^{\mathrm{T}}$$

Repair : (18)

$$\mathrm{PFD}_{\mathrm{CCF}}(t) = \begin{cases} \lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}} T_{\mathrm{P1}}} + (\mathrm{e}^{-\lambda^{\mathrm{UC}} T_{\mathrm{P1}}} - \lambda^{\mathrm{DC}} T_{\mathrm{r}})(\lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}(w_1 - T_{\mathrm{t}})}) \\ \quad \text{for } \{0 \leqslant t - T_{\mathrm{P1}} < \mathrm{TI}\ \&\ T_{\mathrm{t}} \leqslant w_1 < T_{\mathrm{t}} + T_{\mathrm{r}}\} \\ \lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}(T_{\mathrm{P}i} - T_{\mathrm{P}i-1} - T_{\mathrm{t}} - T_{\mathrm{r}})} + (\mathrm{e}^{-\lambda^{\mathrm{UC}}(T_{\mathrm{P}i} - T_{\mathrm{P}i-1} - T_{\mathrm{t}} - T_{\mathrm{r}})} - \lambda^{\mathrm{DC}} T_{\mathrm{r}})(\lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}(w_1 - (T_{\mathrm{P}i} - T_{\mathrm{P1}}) - T_{\mathrm{t}})}) \\ \quad \text{for } \{t - T_{\mathrm{P1}} \geqslant 0\ \&\ T_{\mathrm{P}i} - T_{\mathrm{P1}} + T_{\mathrm{t}} \leqslant w_1 < T_{\mathrm{P}i} - T_{\mathrm{P1}} + T_{\mathrm{t}} + T_{\mathrm{r}}\} \\ \lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}(\mathrm{TI} - (T_{\mathrm{PN}} - T_{\mathrm{P1}}) - T_{\mathrm{t}} - T_{\mathrm{r}})} + (\mathrm{e}^{-\lambda^{\mathrm{UC}}(\mathrm{TI} - (T_{\mathrm{PN}} - T_{\mathrm{P1}}) - T_{\mathrm{t}} - T_{\mathrm{r}})} - \lambda^{\mathrm{DC}} T_{\mathrm{r}})(\lambda^{\mathrm{DC}} T_{\mathrm{r}} + 1 - \mathrm{e}^{-\lambda^{\mathrm{UC}}(w_1 - T_{\mathrm{t}})}) \\ \quad \text{for } \{t - T_{\mathrm{P1}} \geqslant \mathrm{TI}\ \&\ T_t \leqslant w_1 < T_{\mathrm{t}} + T_{\mathrm{r}}\} \end{cases}$$

Standby between tests :

$$\mathrm{PFD}_{\mathrm{CCF}}(t)=\lambda^{\mathrm{DC}}T_{\mathrm{r}}+1-\mathrm{e}^{-\lambda^{\mathrm{UC}}(w_1-(T_{\mathrm{Pi}}-T_{\mathrm{P1}})-T_{\mathrm{t}}-T_{\mathrm{r}})} \quad \text{for}\left\{t-T_{\mathrm{P1}}\geqslant 0\ \&\ \left|\begin{array}{l} T_{\mathrm{Pi}}-T_{\mathrm{P1}}+T_{\mathrm{t}}+T_{\mathrm{r}}\leqslant w_1<T_{\mathrm{Pi+1}}-T_{\mathrm{P1}}\ \text{if}\ i<N \\ T_{\mathrm{PN}}-T_{\mathrm{P1}}+T_{\mathrm{t}}+T_{\mathrm{r}}\leqslant w_1<T_i\ \text{if}\ i=N \end{array}\right.\right\} \tag{19}$$

5.5. Quantification of the average PFD

Observing the example given in Fig. 2, the total PFD(t) for parallel-redundant systems can be calculated with the generalization made in the following equation:

$$\mathrm{PFD}_{\mathrm{TOT}}(t)=\prod_{i=1}^{N}\mathrm{PFD}_{\mathrm{ind}\ i}(t)+\mathrm{PFD}_{\mathrm{CCF}}(t) \tag{20}$$

The point PFD(t_i) can been evaluated in discrete steps (e.g. 1 h) for the length of the mission time. The sum of the point values are then divided over the total number of evaluations n to obtain the $\mathrm{PFD}_{\mathrm{avg}}$

$$\mathrm{PFDavg}\approx\frac{\sum_{i=0}^{T}\mathrm{PFD}(t_i)}{n} \tag{21}$$

The overall model was presented in Torres-Echeverria et al. [43]. Therein, the model was applied to the case study of the system shown in Fig. 2. A sensitivity analysis of the influence of the test policy (TI+TS), the CCF (specifically the β-factor) and the diagnostic coverage was practiced. The application case showed clearly the benefits of staggering the test events of the redundant components of the system, improving the PFD as the time for the test event is distributed more uniformly during the test interval. In addition, increments of testing frequency (lower test intervals) also affected positively the PFD. It was demonstrated that in systems with low-failure-rate components (as it is usual for SIS) the CCF has a significant impact on the system PFD. On the other hand, albeit the diagnostic coverage had as prominent influence as the CCF over the PFD, it certainly improved when the coverage was enhanced. It was shown that using components with better diagnostic coverage reduced the PFD in such a way that it could even be possible to achieve a higher safety integrity level.

6. Optimization of proof testing policies

6.1. The multi-objective proof testing policy problem

As mentioned above, the second objective of this paper is focussed on the optimization of proof testing policies for safety instrumented systems. Given the critical nature of SIS, their optimization must include the three fundamental attributes of safety system: safety integrity, reliability and cost, making it a multi-objective optimization. A multi-objective optimization problem is about simultaneously minimizing the n different objectives of an objective function vector $f(\mathbf{x})$

$$\mathbf{y}=\mathbf{f}(\mathbf{x})=(f_1(\mathbf{x}),f_2(\mathbf{x}),\ldots,f_n(\mathbf{x}))\in\mathbf{Y} \tag{22}$$

where x is a decision variables vector

$$\mathbf{x}=\{x_1,x_2,\ldots,x_n\}\in\mathbf{X} \tag{23}$$

Usually the objective space Y is restricted by a vector of constraints

$$\mathbf{g}(\mathbf{x})=(f_1(\mathbf{x})\leqslant g_1,f_2(\mathbf{x})\leqslant g_2,\ldots,f_n(\mathbf{x})\leqslant g_n) \tag{24}$$

It is generally impossible to find a unique solution that optimizes all the objectives at once. In its place, a set of non-dominated optimal solutions is found, which contain the best solutions in decision space **X**, called the Pareto-optimal set, which constitutes the Pareto-optimal front.

Regarding the specific problem of SIS optimization, the objectives of the objective function vector that represent the three attributes mentioned above are: the system $\mathrm{PFD}_{\mathrm{avg}}$, which determines the SIL of the system, the spurious trip rate (STR) and the lifecycle cost (LCC)

$$\mathbf{y}=\mathbf{f}(\mathbf{x})=(\mathrm{PFD}_{\mathrm{avg}}(\mathbf{x}),\mathrm{STR}(\mathbf{x}),\mathrm{LCC}(\mathbf{x})) \tag{25}$$

As previously mentioned, a proof testing policy includes both the test interval and the test strategy TI+TS. Any change in the policy impacts on the three system's objectives $\mathrm{PFD}_{\mathrm{avg}}$, STR and LCC. The test strategies explored for optimization are the sequential and the staggered test. In previous sections we explained that the difference of staggering can give uniform and non-uniform strategies. Therefore, the degree of distribution of the test events of different components along the test interval of the staggered test strategy provides an additional large number of options for the TS. This distribution is commanded by a factor K (Eq. (27)) that determines the difference of time between the time of first test of the first component (T_{P1}) and the rest of them (Eq. (26)).

Remember in addition that the time elapsed since $t=0$ (system first start-up) up to the first test T_{P} of each component shapes their contribution to the system $\mathrm{PFD}_{\mathrm{avg}}$ during the initial time. The larger the T_{P} the bigger unavailability contribution (recall that in general $\mathrm{PFD}=1-\mathrm{e}^{-\lambda t}$). This initial time interval is bounded by the start of the system being put in service until the periodic tests settle down in a steady state. Therefore the TS is composed by both the T_{P} and the factor K. The T_{P} per component is determined as follows:

$$T_{\mathrm{P1}}=T_{\mathrm{P}},\quad T_{\mathrm{P2}}=T_{\mathrm{P}}+K,\quad T_{\mathrm{P3}}=T_{\mathrm{P}}+2K,\ldots T_{\mathrm{PN}}=T_{\mathrm{P}}+(N-1)K \tag{26}$$

where

$$K=\mathrm{Round}\left(\frac{\mathrm{TI}}{N}P\right) \tag{27}$$

This formulation intends to spread the T_{P}s of the N components along the TI with the same difference of time between them, and that this difference of time is bounded between 0 (in reality T_{t}) and $\mathrm{TI}(N-1)/N$. Note that a higher K (P closer to 1) means higher staggering, while a lower K (P closer to 0) represents lower staggering. This is better explained by an example. Fig. 4 shows the distribution of test events for a three-component system (called A, B and C), and TI = 1 year. The figure illustrates how for $P=0.25$, the three components are tested during the first 2 months of the TI (with $K=1$ month difference between tests). In contrast, when $P=1$, the components are tested with a difference $K=4$ months between them (i.e. they are uniformly staggered, since $K=\mathrm{TI}/N$). Thus, a higher P means a more uniform staggering. It is important to observe that the multiplication factor P (Eq. (27)), which determines the values of K, is the direct decision variable. This means that the testing policy decision variables are TI, T_{P} and P.

In conclusion, it can be said that the for optimization purposes the proof testing policy depends of three variables: test interval TI, time to first test of first component T_{P} and multiplication factor P. Given this, it becomes clear that the decision variable vector per system, which intends to solve the objective function vector in Eq. (23), becomes

$$\mathbf{x}=\{\mathrm{TI},T_{\mathrm{P}},P\} \tag{28}$$

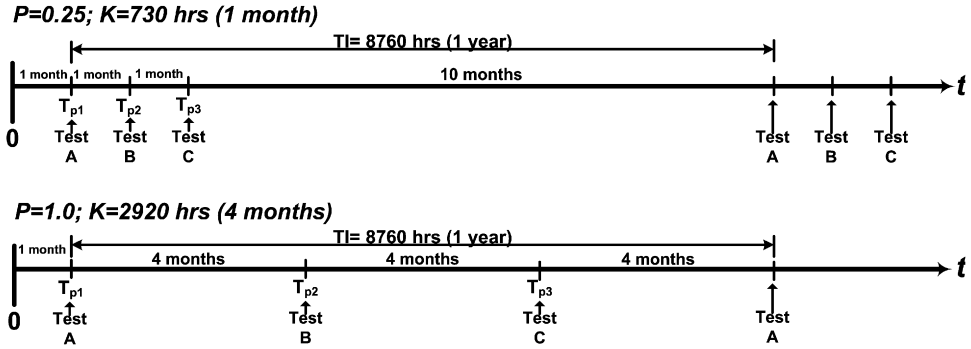


Fig. 4. Example of T_Ps distribution for different K.

6.2. Dependability modelling

The two dependability modelling objectives, PFD_{avg} and STR, are measures affected by two different failure modes, respectively. Therefore, the total failure rate is split into these two modes: dangerous and safe (Eq. (29)). Remember that safe failures contribute to the STR, and dangerous failures to the PFD_{avg}.

$$\lambda^{T} = \lambda^{Danger} + \lambda^{Safe} \tag{29}$$

Each failure mode is in turn further split by the diagnostic coverage and common cause failure of the components into the four sub-failure modes given by Eq. (9). This implies that at the end the total failure rate is split into eight failure modes. This formulation is used here. If the reader is not familiar with this please refer to [1,40] for further explanation.

Quantification of PFD_{avg} is made following the same procedure of Section 5 (Eqs. (20) and (21)). It is, however, very important to remark that the failure modes used for the PFD(t) model (Eqs. (12)–(19)) are only those corresponding to the dangerous failure rates λ^{Danger}. This means that λ^{Danger} substitutes λ^{T} in Eqs. (8) and (9) for feeding the PFD(t) model.

It was mentioned in the introduction that one of the adverse effects of proof testing includes test-induced spurious activations [27]. This is a non-negligible contribution that is usually ignored. The STR model presented here includes it [25]. Therefore, the model for the STR of a component includes the STR caused by internal failures plus the test-induced STR component

$$STR_{T} = STR_{\lambda} + STR_{test} \tag{30}$$

where STR_{λ} is equal to the total STR quantification formula per component given in Ref. [40]. Notice now that the failure modes used for this model are only those corresponding to the safe failure rates λ^{Safe}, which implies that λ^{Safe} substitutes λ^{T} in Eqs. (8) and (9). On the other hand, STR_{test} is estimated by Eq. (31) (developed based on [25], where some guidance can be found to estimate the parameter $P_{r\text{-}trip}$).

$$STR_{test} = \frac{1}{TI} P_{r-trip} = \frac{1}{TI} \frac{N_{trips}}{N_{tests}} \tag{31}$$

Proof testing of every single component of a system can contribute to the test-induced STR. Fig. 5 shows the inclusion of test-induced STR into a fault tree for a two-component system.

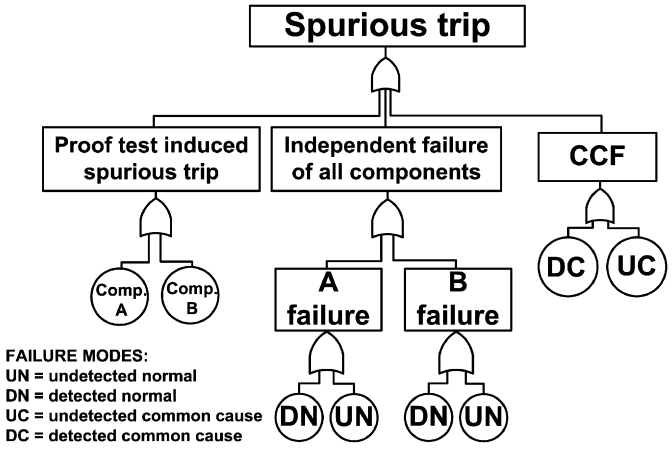


Fig. 5. Fault tree for STR of a two-component system.

6.3. LCC modelling

The lifecycle costs Eqs. (32)–(42) were derived from Ref. [40]. The costs relevant to proof testing include basically the test and corrective maintenance during the system lifetime

$$LCC_{T\&CM} = (C_{T\&CM} + C_{RISK})PVF \tag{32}$$

where

$$C_{T\&CM} = C_{T} + C_{CM} \tag{33}$$

$$C_{T} = \sum_{\forall i} \frac{1}{TI_i} C_i^{T} N_i \tag{34}$$

$$C_{CM} = \sum_{\forall i} f_i^{CM} C_i^{CM} N_i \tag{35}$$

where

$$\sum_{\forall i} f_i^{CM} = \sum_{\forall i} \lambda_i^{T} \tag{36}$$

In this work we are provided with the cost of test per event (alternatively it could also be determined by Eq. (37)). The repair cost is estimated with Eq. (38).

$$C_i^{T} = T_t C_{ht} \tag{37}$$

$$C_i^{CM} = (T_r C_{hr}) + C_{SP} \tag{38}$$

Notice that the corrective maintenance costs include repair of safe failures as well (Eq. (35)). Although proof testing is focussed on dangerous failures detection, it is clear that if safe failures occur they must be repaired as well.

On the other hand, it is important to consider the impact that proof testing has upon the plant risk cost (Eq. (39)). Proof testing has both positive and negative effects upon LCC, since the practice of periodic proof testing reduces the dangerous failure risk when improving PFD_{avg}, but it can increment the safe failure risk (through its adverse impact on the STR).

$$C_{RISK} = C_{STR} + C_{HAZARD} \tag{39}$$

where

$$C_{STR} = STR \times C_{SD} \quad (40)$$

$$C_{SD} = SD_{Time} \times SD_{LOSS} \quad (41)$$

$$C_{HAZARD} = C_{ACC} \times F(ACC|PFD_{avg})PFD_{avg} \quad (42)$$

6.4. Multi-objective GA based on NSGA-II

GA are one technique of evolutionary computation, which mimics the natural selective process of evolution for solution of optimization problems. Different from other optimization methods (e.g. analytical methods), they present several advantages: They are very useful for solving complex, high dimensional, discrete, non-linear and discontinuous problems, with capacity to handle integer variables. GAs are able to deal with problems which objective function is not explicit (such as those generated by fault tree analysis). In addition, they provide the decision maker a pool of optimal solutions. There are many studies related to GAs, since it is a research field in continuous evolution. Good monothematic studies can be found in Refs. [44,45]. For GAs focused on RAMS optimization consult Refs. [46,47].

GAs work with an initial pool of potential solutions called individuals, which constitute the initial population. Each individual is a coded representation of one set of decision variables (a single solution) in the decision space. These individuals are evaluated, selected and mated to create new and hopefully better ones, which are fed into a new generation, making an iterative process that mimics the natural evolution. Each step of the GA is executed through the application of genetic operators.

The standard GA follows generally the following steps:

1. Creation of a random initial population
2. Evaluation of the individuals, which are ranked and fitness assigned
3. Selection for reproduction (crossover), generating a number of new individuals (offspring)
4. Mutation of the offspring
5. Replacement of the offsprings into the original population

The cycle is repeated until a certain condition to stop the algorithm is met. This condition may be the exhaust of a generation count or the compliance with a specific objective. At the end, the algorithm delivers a set of optimal solutions: the Pareto-optimal front.

One of the most competitive algorithms developed so far is the NSGA-II. Ref. [48] stated that it is a highly efficient algorithm, with such a good performance that it has become "a landmark against which other multi-objective evolutionary algorithms have to be compared". Greiner et al. [49] made a comparative study of SPEA2, NSGA-II and NSGA-II with controlled elitism for application to safety system design, and found that the last option gave the best overall average results.

The NSGA-II algorithm was introduced in [50,51]. The controlled elitism addition can be read in [52]. This is a second-generation algorithm. The characteristic feature of this generation is the implementation of elitism in order to preserve the best individuals for subsequent generations. According to Ref. [47], the NSGA-II has the following main features:

- Ranking based on non-domination sorting
- Crowding distance metric as explicit diversity-preserving mechanism

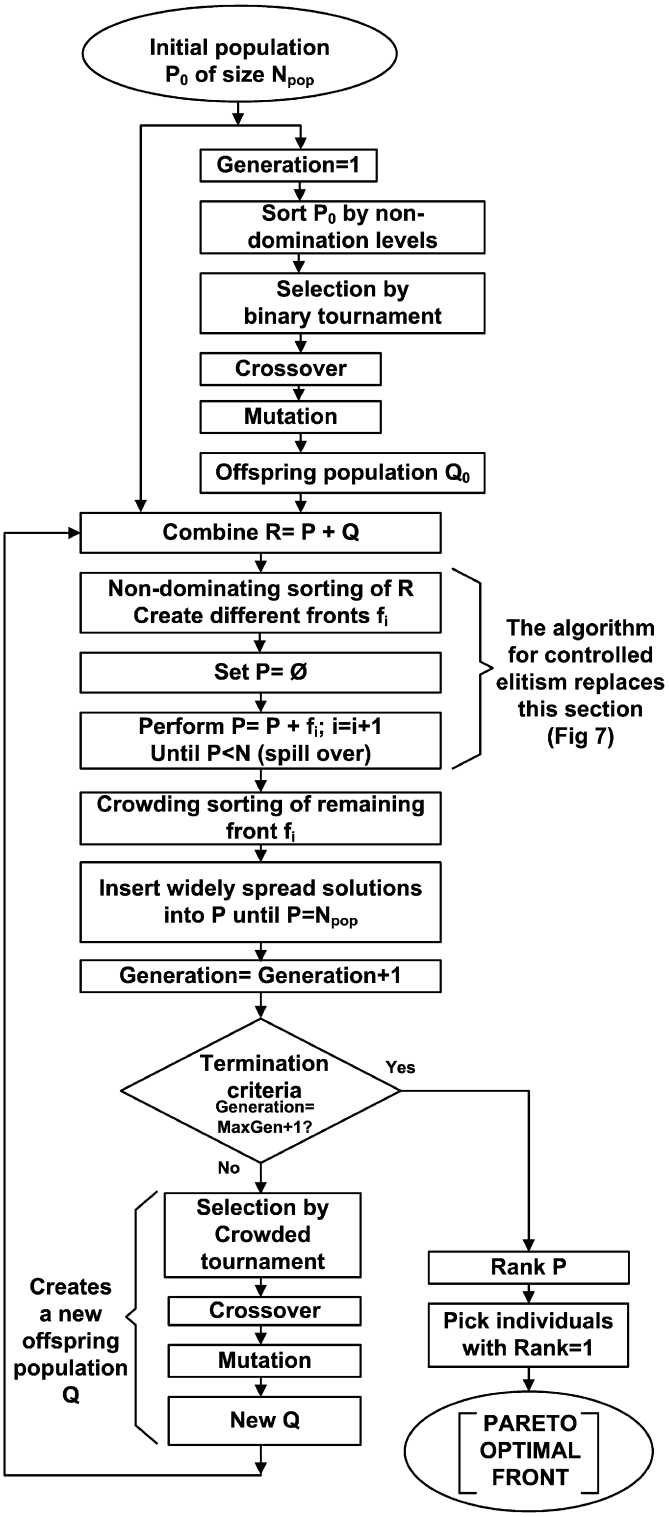


Fig. 6. NSGA-II algorithm.

- Implementation of elitism
- It does not use an external population (as other second-generation algorithm do)
- Optional controlled elitism [52]

A flowchart describing the NSGA-II is shown in Fig. 6. The algorithm works with a population of fixed size N_{pop}. After creating a random initial parent population $\mathbf{P_0}$, an offspring population $\mathbf{Q_0}$, is created based on binary tournament selection.

Both are combined in a population **R** (size $2N_{pop}$). Ranking is then carried out by sorting the individuals by non-domination, creating several different non-dominated fronts. A completely new parent population **P** is generated by adding the individuals from the fronts, starting progressively from the first front (better rank), and so on until a spillover occurs ($\mathbf{P} > N_{pop}$). This is what enforces the elitist mechanism. The entire population of the last front (which spilled over) is sorted by the crowding distance metric (the diversity-preserving mechanism), and the individuals with wider spread (larger crowding distance) are inserted into **P** until $\mathbf{P} = N_{pop}$. This gives as a result the new parent population **P**. From these a new offspring population **Q** is created by crowded tournament selection. By this procedure two solutions are compared, and the one with better rank is chosen; if both have the same rank the one with better crowding distance metric is picked. The new populations **P** and **Q** are then used to feed a new generation (to make $\mathbf{R} = \mathbf{P}+\mathbf{Q}$). The whole procedure is repeated until the termination criterion is met (in Fig. 6 this is a pre-established maximum number of generations). Do not confuse the population array **P** with the multiplication factor P used in Eqs. (27) and (28), since they are completely different things.

The emphasis over elite solutions made twice by NSGA-II can result in solutions not belonging to non-elitist fronts being deleted too soon. This compromises the balance of exploitation versus exploration. This was noticed by Deb and Goel [52]. In order to preserve the good balance, it is necessary to ensure good diversity both along the Pareto-optimal front and also lateral to the front. This latter is called lateral diversity. Deb and Goel conceived a mechanism that intends "to maintain a predefined distribution of the number of individuals in each front" (not only the elitist ones), and thus foster lateral diversity (at the time the NSGA-II promotes diversity along the front). This is made by restricting "the number of individuals in the current best non-dominated front adaptively". Controlled elitism is introduced into the NSGA-II algorithm for generating a new parent population **P** during each generation. Fig. 7 shows the controlled elitism algorithm flowchart. Fig. 6 provides an indication how it is introduced into the NSGA-II algorithm. The parameter r (which is <1) helps to predetermine the number of individuals n_i from each front that can be inserted into the population P. The entire description of the algorithm can be found in the mentioned reference.

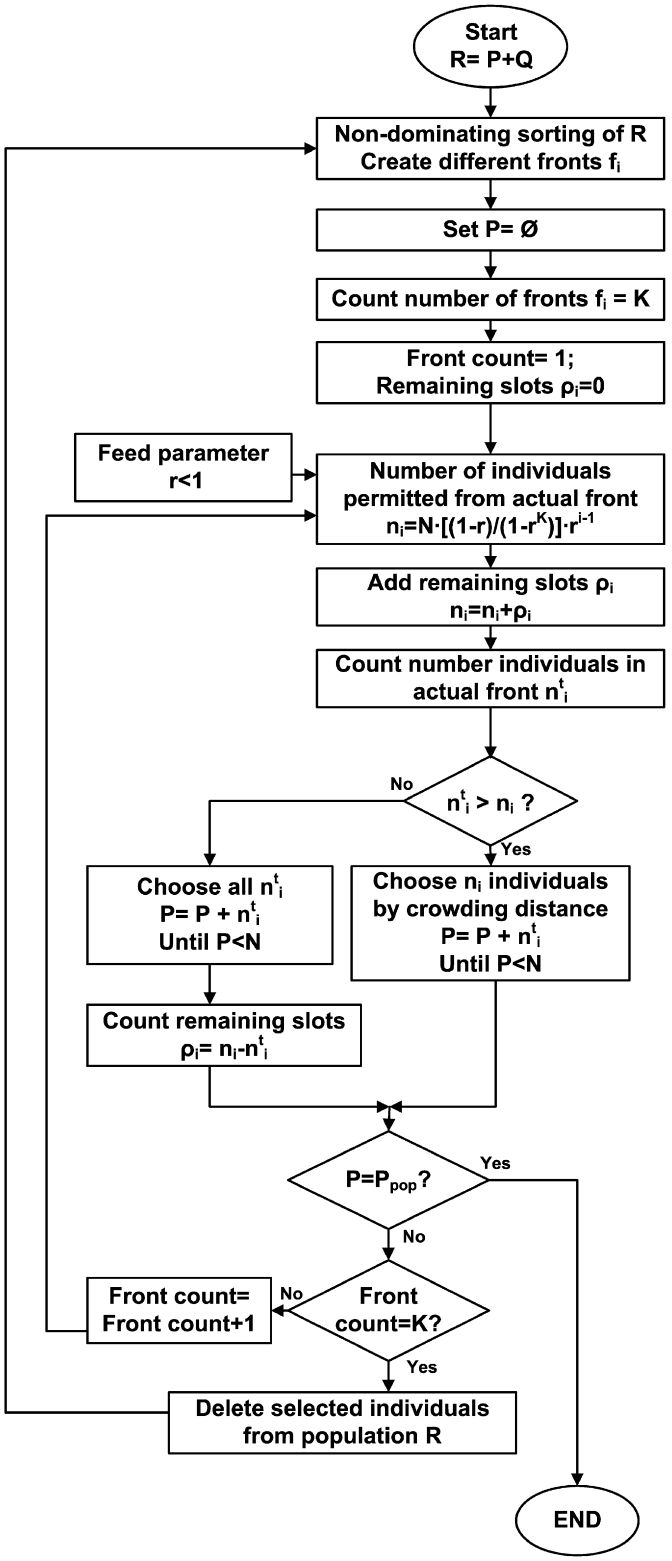


Fig. 7. Controlled Elitism algorithm.

7. Application case

7.1. Description of the problem and approach

In order to illustrate the overall approach, the proof test optimization is applied to a protection system against high pressure and temperature of a chemical reactor (Fig. 8). The

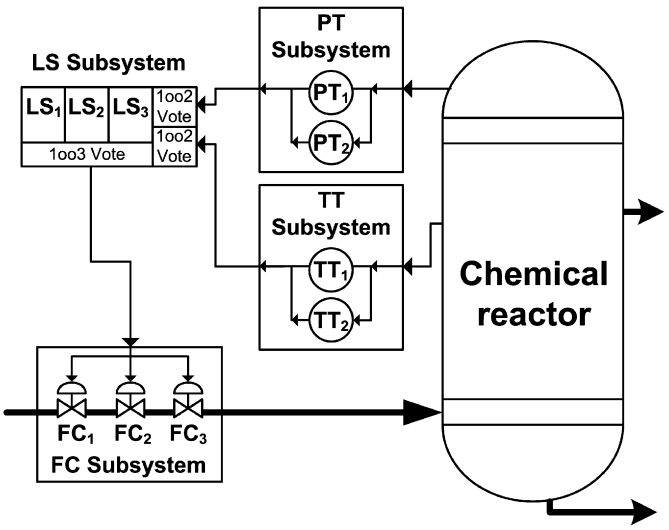


Fig. 8. Chemical reactor protection system.

Table 2
Dependability and lifecycle cost data.

Instrument	Safety PLC	Standard PLC	Electronic PT	Press. switch	Smart TT	Temp. switch	Air Opt. valve
λ^{Safe} ($\times 10^{-6}$/h)	3.46	3.94	2.16	6.81	5.05	9.22	3.94
λ^{Danger} ($\times 10^{-6}$/h)	0.0036	4.81	1.90	4.11	0.348	7.60	3.35
λ^{Total} ($\times 10^{-6}$/h)	3.492	8.75	4.06	10.92	5.398	16.82	7.29
ε^{Safe} (%)	100	45	56	10	100	10	00
ε^{Danger} (%)	81.25	60	51.1	10	7.5	10	25
$C^{purchase}$ ($)	3000	2000	2300	500	2560	500	6940

Lifecycle cost data:
Design/install/commissioning PLC = 10,320 ($)
Maintenance PLC = 960 ($/event)
Test PLC = 240 ($/event)
Design overall instrumentation = 3060 ($)
Installation/commissioning per instrument = 600 ($)
Maintenance per instrument = 240 ($/event)
Test per instrument = 60 ($/event)
Repair cost per instrument and PLC = 60 ($/h)
Spares per repair = 25% component cost per event
Cost safety PLC rack: 31000 ($)
Cost standard PLC rack: 20500 ($)
Loss of production = 2000 ($/h)
Start up cost = 1800 ($)
Catastrophic loss = 150×10^6 ($)
SIS operational life = 15 (years)
Discount rate = 0.05

Other data:
$\beta = 10\%$ (for all instruments, both failure modes)
$\beta = 5\%$ (smart TT, safe failure only)
$T_r = 8$ (h)
$T_t = 1$ (h)
$C_{1oo2} = 1.0$
$C_{1oo3} = 0.3$
Shut down time = 24 (h)
Plant risk without SIS = 8.55 ($\times 10^{-3}$/yr)

system is composed of four subsystems: Temperature transmitter (TT), pressure transmitter (PT), logic solver (LS) and final control element (FC). Upon detection of either high temperature or pressure the safety system cuts the reactor supply off in order to prevent a runaway reaction. Each subsystem is parallel redundant. All data are presented in Table 2.

The optimization cases presented here consider the possibility of using sequential or staggered test. As concluded above, partial testing provides the opportunity for optimization. International standards do not present constraints for partial testing, in addition that they do not provide sound guidance about the setting up of TIs. Therefore, the proof test optimization problem is addressed by optimizing the test policy for each single subsystem. This means that 12 variables (3 per subsystem: TI, T_P and P) compose the decision variables vector in Eq. (28).

Three different cases have been analyzed:

- *Case* 1. SIS with safety instruments: Safety PLC, conventional electronic pressure transmitter, smart temperature transmitter and air-operated valve. We use a $P_{r\text{-}trip} = 1 \times 10^{-3}$. Kim et al. [24,25] suggest a method for estimation of $P_{r\text{-}trip}$, giving even examples for some instruments. The value used here has been estimated as a representative average following values provided by those references.
- *Case* 2. The SIS uses the same kind of instruments, but with a $P_{r\text{-}trip} = 0$ (ideal proof test case), in order to draw comparisons with Case 1.
- *Case* 3. The SIS is composed by standard instruments: Standard PLC, electromechanical switches for pressure and temperature and air-operated valve: $P_{r\text{-}trip} = 1 \times 10^{-3}$.

The following assumptions apply: The system operates in low-demand mode. Components of each subsystem are independent and identical, with constant failure rate. Once a component has failed it remains in that state until it is repaired. Testing and repair are perfect: no faults overlooked, and the item is returned to an "as new" condition or normal state after repair. Only the CCF that affects all redundant components is considered. It is assumed that test and repair times (T_t and T_r) are the same for all components.

7.2. Problem modelling

Modelling of the three objectives has been discussed in Section 6, so only some minor problem-specific details are added here. System dependability is quantified by fault tree analysis. The fault tree for STR follows the philosophy shown in Fig. 5. The fault tree for PFD is presented in Fig. 9. Eq. (43) gives the expression that calculates the system PFD(t). Each term in the equation indicates correspondence to a subsystem by a subscript. The PFD(t) is calculated every hour during the system's operating life (mission time). Then, the PFD_{avg} is obtained by averaging the point values as indicated in Eq. (21).

$$PFD(t) = PFD_{LS}(t) + (PFD_{PT}(t) \times PFD_{TT}(t)) + PFD_{FC}(t) \quad (43)$$

7.3. Implementation of the genetic algorithm

The optimization is based on the NSGA-II (Fig. 6). All the 12 decision variables of the problem are integer. Explicit constraints are included in the decision vector: the TI is bounded between 30 and 365 days (730 for the LS), and T_Ps between 0 and TI. The multiplication factor P is a number between 0 and 1000 (then multiplied by 1×10^{-3} so that $P < 1$). The total number of potential solutions is 9.27×10^{29}. Given that the decision variables are integer, an integer code is used rather than the classic binary code. This opened the possibility of using genetic operators for both binary and real numbers (with minor modifications), especially for the crossover and mutation operators.

A baseline algorithm setup was used as initial configuration. Then the algorithm was tuned trying several different schemes. A qualitative comparison was made between pairs of different algorithm setups, based on a good balance of the following factors: convergence (proximity to the real (unknown) Pareto-optimal front), and the diversity along the front (good uniform spread of solutions) and number of individuals in the optimal set.

Few of the GA multi-objective optimization studies report use of used integer codification. They implement combinations of operators for either binary or real numbers. As previously

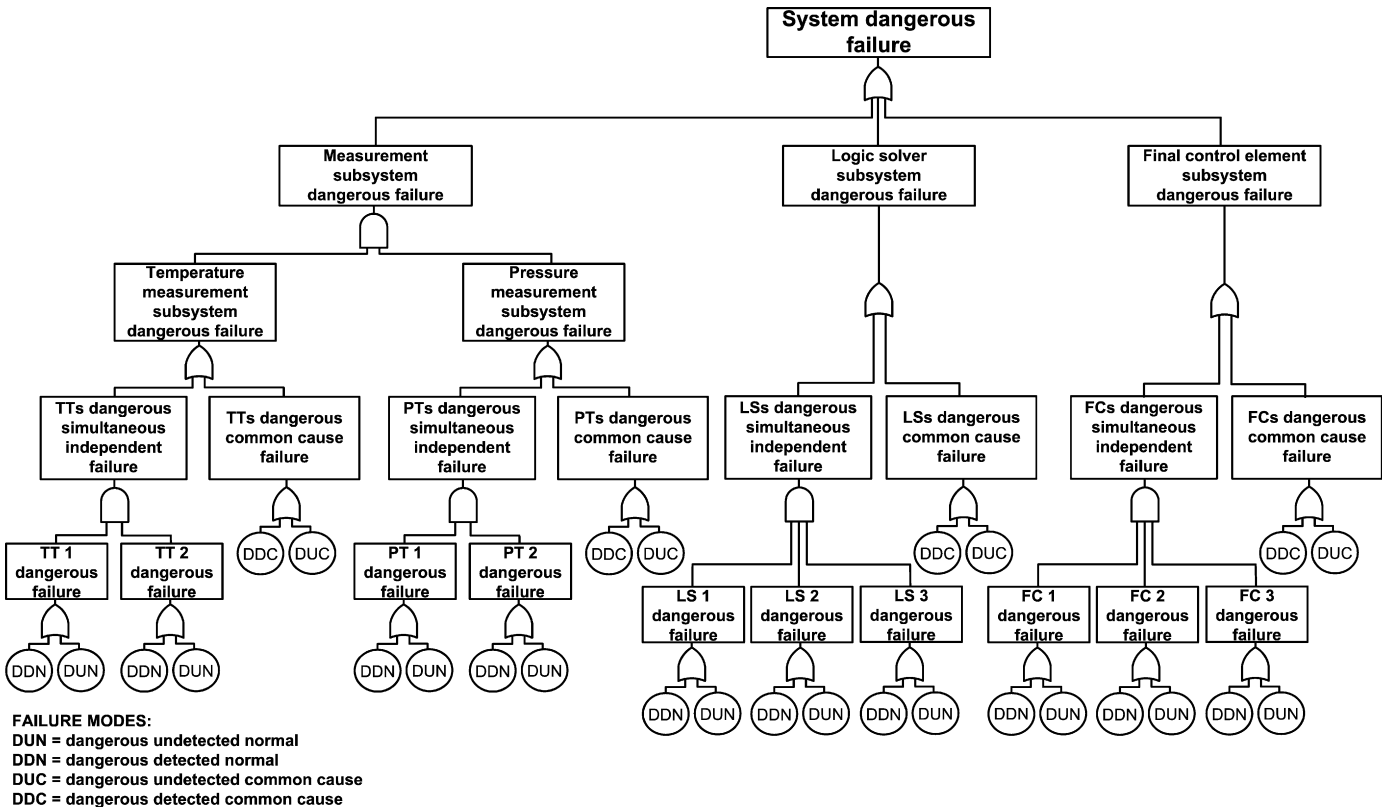


Fig. 9. Fault tree for quantification of PFD.

mentioned, the difference basically resides on the crossover and mutation operators. In general, the operators for binary codes used for integers are single, double and multiple-point crossover. Integer mutation through a method for binaries is basically similar to bit flipping (in this case integer flipping: just replacing randomly one integer by a new one), also called random replacement. In contrast, methods for real codes are more sophisticated. Crossover is made either by uniform recombination (also called discrete recombination, in fact an adaptation of multiple-point crossover), or by use of algebraic formulae that combine the values of the two parent variables into a new variable value (this technique intends to enhance the introduction of new genetic material). These are a group of algorithms generally referred to under the generic name of Blending methods [45], although some other denominations specific to each algorithm are found [53]; e.g. intermediate recombination, line recombination, etc. Regarding mutation operators, one method is to add a normally distributed random number (bounded between a specific range) to a variable. There is not specific name for this kind of algorithm so we simply call it "mutation for real numbers".

Billings and Zheng [54] reported what seems to be two variations of the uniform crossover algorithm, called fixed length and variable length crossover. Mutation was made through a mechanism similar to integer flipping. Weile and Michielsen [55] proposed special formulas for crossover (similar to blending methods), and utilized addition or subtractions formulas for mutation. Tao et al. [56] implemented double-point crossover, and performed mutation through addition formulas. Damousis et al. [57] used multiple-point crossover and employed a non-uniform mutation operator. Li et al. [58] applied formulas similar to blending methods for crossover, and mutated the individuals through integer flipping. Some other optimization studies were found, but they do not specify which operators they implement.

The settings that were tried (in different combinations) are detailed below. The crossover and mutation operators for real number that were tested can be found in Ref. [59].

- *Population size and number of generations*: 50/100, 100/50, 200/20, 100/25 and 100/40.
- *Mutation*: Flip mutation at 0.1 and 0.2; mutation for real numbers at 0.1 and 0.2.
- *Recombination*: Single point crossover, double-point crossover, discrete recombination (uniform crossover) and blending crossover [59].
- *Selection*: Basic NSGA-II against controlled elitism at $r = 0.50$, 0.80, 0.65 and 0.40.

In general, it can be reported that a population size larger than 100 provided no significant improvement. Fine tuning could be made through changing the number of generations. Changing the mutation operator (using both algorithms for real and binary codes) did not make any important difference. From all the recombination operators compared, discrete recombination showed a slightly better performance, while a more notable improvement was found with the blending crossover: Better distribution along the optimal front and enhanced exploration since new solutions at the two extremes of the front were found. The most significant enhancement overall was achieved by implementing controlled elitism. It was noticed that this improves the exploration made by the algorithm. Nevertheless, the exploitation can be compromised, and it is only improved if the right reduction rate parameter r is found. This fact, however, increments the complexity of the NSGA-II implementation, since the parameter r has to be tuned. The improvements achieved in terms of exploration meant that both ends of the optimal front were further pushed, finding new solutions. The distribution along the front was slightly enhanced as well.

The best final combination of parameters was:

- Population size: 100 individuals.
- Number of generations: 50.
- Recombination by blending crossover at 0.7.
- Flip mutation at 0.1.
- Selection using controlled elitism with $r = 0.40$.

8. Results and discussion

Case 1 is taken as the baseline case. The results of this case are presented in Fig. 10. In order to demonstrate the benefits of the optimization by GA, the Pareto-optimal front (i.e. optimal set) is compared against the initial population. A specific single solution may have been proposed as an initial candidate solution and introduced into the initial population for comparisons purposes. However, it was considered that the random creation of the initial population (initial set) would contain both good and bad solutions. So this is used as basis for comparison. It is easily observed that the optimal set is superior to the initial set in every objective. The optimal set provides a smooth well-distributed set of solutions for the decision maker to choose. It is clear that every single solution provided in the initial set is dominated by the optimal set. Therefore, the advantage of implementing the GA-based optimizer is undoubtable. The plots in Fig. 10 (right) compare the performance or both populations by pairs of objectives. Notice that for any PFD_{avg} value, the cost (LCC and STR) in the optimal set is always lower than in the initial set. It shows the benefits of risk reduction comparing again both initial and optimal sets. It can be seen how around solution 50, any reduction in PFD_{avg} has a lower LLC and STR costs (in comparison with the initial set). After that point, further reduction of the PFD_{avg} is more expensive. Notice, however, that in every case the PFD_{avg} is lower for the optimal set than for the initial set. Observe though how when the graph is following the increasing solution numbers, gains in terms of PFD_{avg} become more expensive in terms of both LCC and STR. The optimal set shows that there is an area where risk reduction (lower PFD_{avg}) is practically free (low-cost increment). Around solution 38, risk reduction becomes much more expensive.

Fig. 11 shows the Pareto-optimal fronts for the three cases. The complete results obtained from all three cases are presented in Fig. 12. This figures plot the multiplication factor P rather than K, since it is more representative of the degree of distribution of the test events along the test interval of component (see Eq. (27)). It is very important to notice that a different scale is used for Case 3 in comparison to Cases 1 and 2. Observe the two scales at the top of Fig. 12, at the left and right extremes.

Fig. 11 demonstrates that the PFD_{avg} is consistently in conflict with the STR and LCC. In addition, notice that in Case 1 there is a saturation point in the Pareto-optimal front, where a marginal reduction of PFD_{avg} causes a large increment of both STR and LCC. This starts around solution 80 (seen in Fig. 12). Trying to achieve any further small reduction of PFD_{avg} after this point is very expensive, and it would be justified only to meet some specific safety requirements. Notice that from solution 1 to 41 risk reduction cost is more profitable (the cost of improving the PFD_{avg} is less significant).

Analyzing the decision variables obtained for the optimal set of Case 1, the following comments can be drawn. First of all, the test interval TI of every component depends on its reliability specifications. The lower the dangerous failure rate (and higher diagnostic coverage), the higher TI (lower the test frequency). Observe that, in general, the TI for the LS subsystem is above 700 days, while for the other subsystems it is below 400 days.

On the other hand, for the LS, PT and TT subsystems the T_{P1} and P variables seem not to have a consistent trend. However, the case of the shutdown valves (FC) is notable. Firstly, the T_{P1} is generally low, meaning they are tested early from the start of the system's

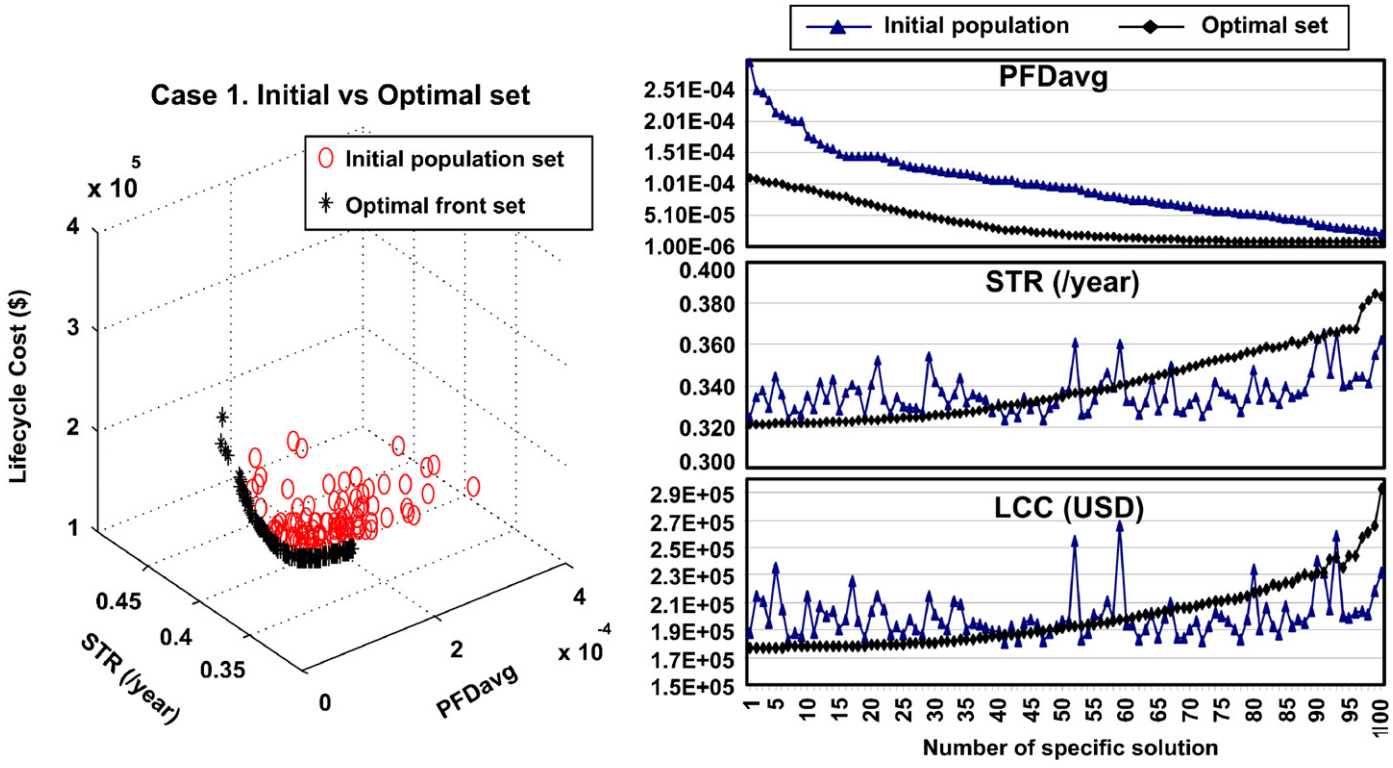


Fig. 10. Case 1. Comparison of initial and optimal set populations.

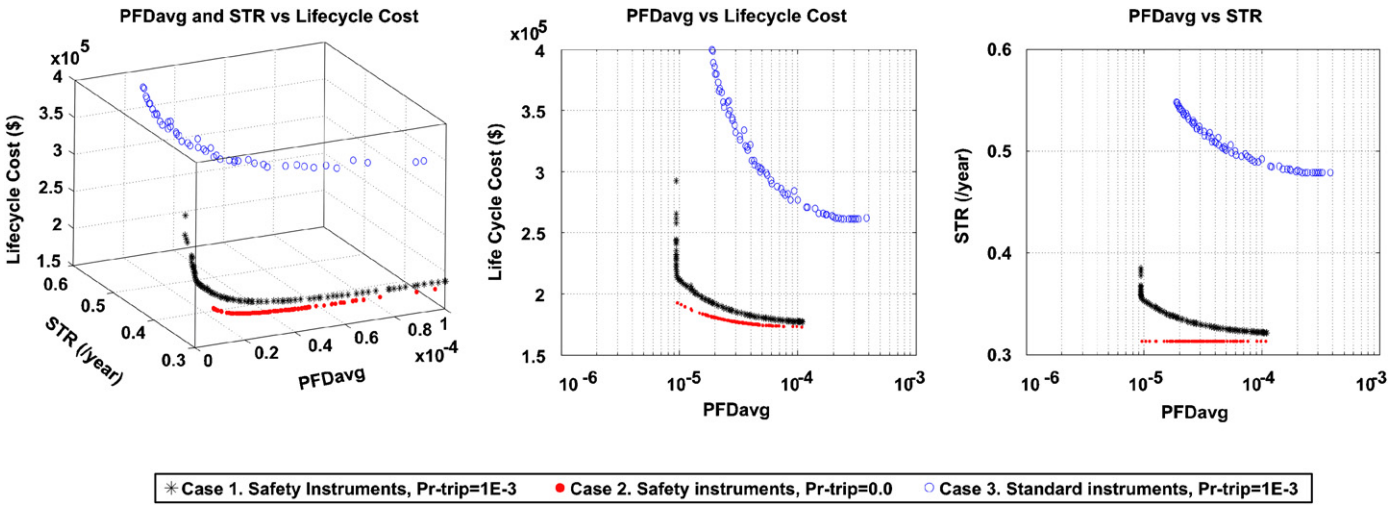


Fig. 11. Comparison of the optimal front for the three cases.

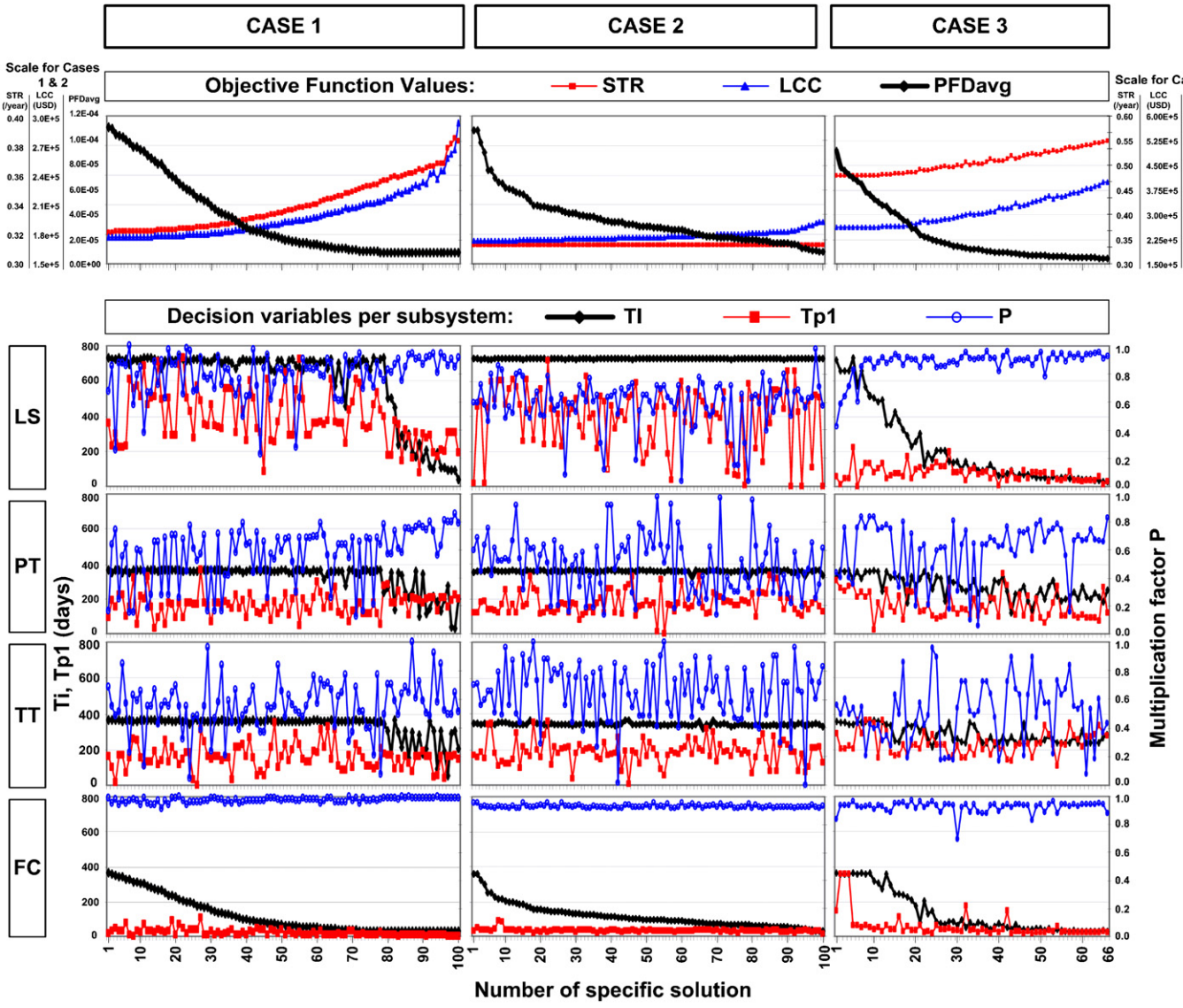


Fig. 12. Results obtained from all cases.

operational life. The FC subsystem TI declines steadily along the system PFD_{avg} reduction; i.e. the overall system PFD_{avg} is following the FC's TI trend. This is due to the fact that the FC subsystem is the system's weakest link, given that the valves have the highest failure rate and lowest diagnostic coverage. If we follow the parallel decline of system PFD_{avg} together with the FC's TI, we will notice that once the TI is very low, and cannot be further lowered, the TI of the other subsystems starts decreasing, i.e. lower TI, (around solution 80), specially in the LS. This indicates that the improvements in PFD_{avg} are achieved testing the FC components more frequently. Once this frequency cannot be increased, the benefits have to be extracted increasing the test frequency of another subsystem. This is by natural choice the LS, because the TP and TT subsystems act as redundant subsystems of one another. Also observe that in both the FC and LS subsystems, once the TI has been considerably lowered, the T_{P1} (time to first test) is also decreased.

Note that the behaviour of the multiplying factor P is similar to the one of TI. This means that in order to obtain further reductions the distribution of the test of the components is made more uniformly; i.e. full staggering. Notice that no single value of P goes close to zero, which clearly indicates that sequential testing is completely discarded.

Observe now the performance of the optimization for Case 2 in Fig. 12. Remember that Case 2 corresponds to the ideal case were the proof testing activity has not adverse affects over the system STR ($P_{r\text{-}trip} = 0$). In the upper plot is noticed that PFD_{avg} is still in conflict with the LCC: the safer the system the more expensive. However, it is not in conflict with the STR anymore. Observe the STR plot is flat. Different from Case 1, there is no saturation point; the test frequency is constantly low for the three more dependable subsystem: LS, PT and TT. Nevertheless, the PFD_{avg} behaviour continues to follow the FC's TI trend: the shutdown valves remain the system weakest link. Referring to Fig. 11, it is also evident that the LCC is lower. This situation is possible basically because there is no STR increment caused by testing. Thus, the most important conclusion from comparing Case 1 against 2 can be drawn: Apart from the natural costs caused by the testing activity itself, it is illusory not to consider the adverse effects of testing, especially for STR in safety systems. Testing does have an additional, considerable cost through STR. Therefore, it is fundamentally important to consider its adverse effects for decision making.

Finally, consider the results given by Case 3 in Fig. 12. This case implements the safety function using less reliable components than in Case 1. Notice the difference between Cases 3 and 1 in the optimal solutions obtained (do not forget to consider their different scales again). The most notable is the case of the LS subsystem. All in all, now the system PFD_{avg} follows both, the LS and FC test interval. This is because now the PLCs failure rates (and diagnostic coverage) are not outstandingly lower then the rest of the instruments (as it was in Case 1). Notice as well that the subsystems PT and TT are tested more frequently than in Case 1, despite they back up each other. Fig. 11 clearly shows that the optimal set of Case 3 is always dominated by the one of Case 1. At every moment, the cost for a similar PFD_{avg} is higher for Case 3, both in terms of STR and LCC. The acquisition cost for Case 3 system may be lower, but in the long term, for the entire system's operating life, Case 1 is much less costly. In addition, the subsystems PT and TT are tested more frequently than in Case 1, despite the fact that they are backing up each other.

9. Concluding remarks

This paper approached the modelling and optimization of proof testing policies for safety instrumented systems (SIS). A model with a new approximation for the time-dependent PFD of parallel architectures in SIS is presented here. The model addresses the level of modelling detail required by IEC 61508, since it is capable of evaluating explicitly the effects of the component's diagnostic coverage and the system common cause failure. It is powerful enough to accommodate different testing strategies, i.e. simultaneous, sequential and staggered (with varying degrees of staggering).

The model has been integrated into a methodology for optimization of proof testing policies with the NSGA-II genetic algorithm. The requirements of IEC 61508 and current real testing practice for SIS are taken into account. Thus, the relevant level of detail is addressed in the dependability models. This makes it a very a practical approach. Proof testing system policies are evaluated as the combination of test interval and strategy TI+TS. The test strategy includes the time for first test of the components the strategy itself (sequential or staggered), and the level of staggering. The optimization has been based on very relevant objectives for safety-critical systems: PFD_{avg}, STR and LCC.

The results obtained show that proof testing is very relevant for achieving and maintaining high levels of system safety integrity. Increments of testing frequency (lower test intervals) generally affect positively the PFD_{avg}. This is, however, in conflict with the system lifecycle cost. Apart from the obvious benefits of the testing activity, it certainly conveys also adverse effects. One of the most relevant is an increment of the STR. It has been demonstrated that ignoring this factor in the dependability model results in overoptimistic assumptions. It is therefore important to consider and quantify them. On the other hand, the results show that the combination TI+TS is very influential on the system safety integrity, especially the test intervals of the components with the lowest dependability in the system (high dangerous failure rates and low diagnostic coverage) have even shaped the system PFD_{avg}. The second observed factor is the trend of the test to be fully staggered for lower PFD_{avg} levels. The application case showed the benefits of staggering the test events of the redundant components of the system, improving the PFD_{avg} as the time for the test event is distributed more uniformly during the test interval.

The choice of instruments with lower cost may lower the initial acquisition cost of the system, but at the same time they can have the effect of increasing considerably the overall system lifecycle cost. This knowledge can be used for decision making regarding system design at early stages of the lifecycle. Nevertheless, testing should not be overexploited. There is one limit point where further intensification of the testing activity certainly enhances safety lowering the PFD_{avg} down but a very high cost: Very small marginal decrements of PFD_{avg} may have disproportionate costs in terms of both spurious trip rate and lifecycle cost. Considering all these factors, it is easy to conclude that the optimization methodology implemented offers many benefits to the decision maker, in terms of both the optimization itself and a better knowledge of the safety system behaviour, making it a powerful tool analysis for practical industrial application.

References

[1] Goble WM. Control systems safety evaluation & reliability. USA: The Instrumentation, Systems and Automation Society; 1998.
[2] International Electrotechnical Commission. IEC 61508. Functional safety of electrical/electronic/programmable electronic safety-related systems. Parts 1–7, Switzerland, 1998–2005.
[3] The Instrumentation, Systems and Automation Society. ISA TR84.0.02. Safety instrumented systems. Safety integrity level evaluation techniques. Draft version 5, Parts 1–5, USA, 1999.
[4] Hauge S, Hokstad P, Langseth H, Oien K. Reliability prediction method for safety instrumented systems. PDS method handbook 2006 edition. Norway: SINTEF; 2006.

[5] Martorell SA, Melia ME, Marti SM, Garcia JS, Soriano-Melchor F, Verdu-Martin G, et al. Análisis del periodo de pruebas optimo en la revisión de especificaciones técnicas de C.N. relativas a STI's. Aplicación a un sistema por dos componentes en paralelo. In: XIV annual meeting of the Spanish Nuclear Society. Marbella, Spain, 1988.
[6] Martorell SA, Serradell VG, Samanta PK. Improving allowed outage time and surveillance test interval requirements: a study of their interactions using probabilistic methods. Reliab Eng Syst Saf 1995;47(2):119–29.
[7] Cepin M, Mavko B. Probabilistic safety assessment improves surveillance requirements in technical specifications. Reliab Eng Syst Saf 1997;56(1): 69–77.
[8] Cepin M. Optimization of safety equipment outages improves safety. Reliab Eng Syst Saf 2002;77(1):71–80.
[9] International Electrotechnical Commission. IEC 61511. Functional safety—safety instrumented systems for the process industry sector. Parts 1–3, Switzerland, 2003.
[10] The Norwegian Industry Association. OLF-070. Application of IEC 61508 and IEC 61511 in the Norwegian Petroleum Industry, Norway, 2004.
[11] The Instrumentation, Systems and Automation Society 1996. ISA S84.01-1996. Application of safety instrumented systems for the process industry, USA, 1996.
[12] American Petroleum Institute. API RP 14C. Recommended practice analysis, design, installation, and testing of basic surface safety systems for offshore production platforms, 7th ed. Washington, DC, USA, 2001.
[13] Samanta P, Kim IS, Mankamo T, Vesely WE. Handbook of methods for risk-based analyses of technical specifications, NUREG/CR-6141. USA: Nuclear Regulatory Commission; 1994.
[14] Health and Safety Executive. Principles for proof testing of safety instrumented systems in the chemical industry. Contract research report 428/2002. Sheffield, UK, 2002.
[15] Uryas'ev S, Vallerga H. Optimization of test strategies: a general approach. Reliab Eng Syst Saf 1993;41(2):155–65.
[16] Schofield SL. Optimisation of proof test intervals in fault tree analysis. Reliab Eng Syst Saf 1993;41(2):127–33.
[17] Vaurio JK. The theory and quantification of common cause shock events for redundant standby systems. Reliab Eng Syst Saf 1994;43(3):289–305.
[18] Vaurio JK. Optimization of test and maintenance intervals based on risk and cost. Reliab Eng Syst Saf 1995;49(1):23–6.
[19] Hokstad P, Flotten P, Holmstrom S, McKenna F, Onshus T. A reliability model for optimization of test schemes for fire and gas detectors. Reliab Eng Syst Saf 1995;47(1):15–25.
[20] Bukowski JV. Modeling and analyzing the effects of periodic inspection on the performance of safety-critical systems. IEEE Trans Reliab 2001;50(3):321–9.
[21] Vaurio JK. Common cause failure probabilities in standby safety system fault tree analysis with testing-scheme and timing dependencies. Reliab Eng Syst Saf 2003;79(1):43–57.
[22] Lapa CMF, Pereira CMNA, Frutuoso e Melo PF. Surveillance test policy optimization through genetic algorithms using non-periodic intervention frequencies and considering seasonal constraints. Reliab Eng Syst Saf 2003; 81(1):103–9.
[23] Courtois PJ, Delsarte P. On the optimal scheduling of periodic tests and maintenance for reliable redundant components. Reliab Eng Syst Saf 2006; 91(1):66–72.
[24] Kim IS, Martorell S, Vesely WE, Samanta PK. Quantitative evaluation of surveillance test intervals including test-caused risks. NUREG/CR-5775, BNLNUREG-52296. New York, USA: Brookhaven National Laboratory; 1992.
[25] Kim IS, Martorell S, Vesely WE, Samanta PK. Risk analysis of surveillance requirements including their adverse effects. Reliab Eng Syst Saf 1994;45(3): 225–34.
[26] Zhao J, Chan AHC, Roberts C, Madelin KB. Reliability evaluation and optimization of imperfect inspections for a component with multi-defects. Reliab Eng Syst Saf 2007;92(1):65–73.
[27] Lundteigen MA, Rausand M. Spurious activation of safety instrumented systems in the oil and gas industry: basic concepts and formulas. Reliab Eng Syst Saf 2008;93(8):1208–17.
[28] Pattison RL, Andrews JD. Genetic algorithms in optimal safety system design. Proc IMechE PtE; J Proc Mech Eng 1999;213:187–97.
[29] Martorell S, Carlos S, Sanchez A, Serradell V. Constrained optimization of test intervals using a steady-state genetic algorithm. Reliab Eng Syst Saf 2000; 67(3):215–32.
[30] Marseguerra M, Zio E. Optimizing maintenance and repair policies via combination of genetic algorithms and Monte Carlo simulation. Reliab Eng Syst Saf 2000;68(1):69–83.
[31] Busacca PG, Marseguerra M, Zio E. Multiobjective optimization by genetic algorithms: application to safety systems. Reliab Eng Syst Saf 2001;72(1): 59–74.
[32] Martorell S, Sanchez A, Carlos S, Serradell V. Simultaneous and multi-criteria optimization of TS requirements and maintenance at NPPs. Ann Nucl Energy 2002;29(2):147–68.
[33] Andrews JD, Bartlett LM. Genetic algorithm optimisation of a firewater deluge system. Qual Reliab Eng Int 2003;19(1):39–52.
[34] Vinod G, Kushwaha HS, Verma AK, Srividya A. Optimization of ISI interval using genetic algorithms for risk informed in-service inspection. Reliab Eng Syst Saf 2004;86(3):307–16.
[35] Martorell S, Sánchez A, Carlos S, Serradell V. Alternatives and challenges in optimizing industrial safety using genetic algorithms. Reliab Eng Syst Saf 2004;86(1):25–34.
[36] Martorell S, Villanueva JF, Carlos S, Nebot Y, Sánchez A, Pitchard JL, et al. RAMS+C informed decision-making with application to multi-objective optimization of technical specifications and maintenance using genetic algorithms. Reliab Eng Syst Saf 2005;87(1):65–75.
[37] Martorell S, Carlos S, Villanueva JF, Sanchez AI, Cepin M. Optimization of surveillance requirements at NPP considering time-dependent scheduling of tests and preventive maintenance. In: Kolowroki K, editor. Proceedings of the European safety and reliability conference ESREL '05. Tri City, Poland, 2005. p. 1363–70.
[38] Martorell S, Carlos S, Villanueva JF, Sanchez AI, Galvan B, Salazar D, et al. Use of multiple objective evolutionary algorithms in optimizing surveillance requirements. Reliab Eng Syst Saf 2006;91(9):1027–38.
[39] Torres-Echeverria AC, Thompson HA. Multi-objective genetic algorithm for optimization of system safety and reliability based on IEC 61508 requirements: a practical approach. Proc IMechE PtO: J Risk Reliab 2007;221(O3): 193–205.
[40] Torres-Echeverria AC, Martorell S, Thompson HA. Design optimization of a safety instrumented system based on RAMS+C addressing IEC 61508 requirements and diverse redundancy. Reliab Eng Syst Saf 2008.
[41] Cepin M. Sequential versus staggered testing towards dynamic PSA. In: Proceedings of the second regional meeting on nuclear energy in Central Europe. Portoroz, Slovenia, 1995. p. 184–9.
[42] Mosleh A, Fleming KN, Parry GW, Paula HM, Worledge DH, Rasmuson DM. Procedures for treating common cause failures in safety and reliability studies. NUREG/CR-4780 (EPRI NP-5613). USA: Nuclear Regulatory Commission; 1988.
[43] Torres-Echeverria AC, Martorell S, Thompson HA. Modelling test strategies effects on the probability of failure on demand for safety instrumented systems. In: Martorell S, Guedes-Soares C, Barnett J, editors. Proceedings of the European safety and reliability conference ESREL '08 and 17th ESRA-Europe. Valencia, Spain, 2008. p. 1881–9.
[44] Deb K. Multiobjective optimization using evolutionary algorithms. Chichester, UK: Wiley; 2001.
[45] Haupt RL, Haupt SE. Practical genetic algorithms. New Jersey, USA: Wiley; 2004.
[46] Marseguerra M, Zio E, Martorell S. Basics of genetic algorithms optimization for RAMS applications. Reliab Eng Syst Saf 2006;91(9):977–91.
[47] Konak A, Coit DW, Smith AE. Multi-objective optimization using genetic algorithms: a tutorial. Reliab Eng Syst Saf 2006;91(9):992–1007.
[48] Coello-Coello CA. Evolutionary multi-objective optimization: a historical view of the field. IEEE Comput Intell Mag 2006;1(1):28–36.
[49] Greiner D, Galvan B, Winter G. Safety systems optimum design by multicriteria evolutionary algorithms. In: Fonseca CM, et al., editors. Proceedings of the evolutionary multi-criterion optimization conference, Faro, Portugal, 2003. Lecture Notes in Computer Science no. 2632. London: Springer; 2003. p. 722–36.
[50] Deb K, Agrawal S, Pratab A, Meyarivan T. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In: Marc Schoenauer, et al., editors. Proceedings of the parallel problem solving from nature sixth conference. Paris, France, 2000. Lecture Notes in Computer Science no. 1917. London: Springer; 2000. p. 585.
[51] Deb K, Agrawal S, Pratab A, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evolutionary Comp 2002;6(2):182–97.
[52] Deb K, Goel T. Controlled elitist non-dominated sorting genetic algorithms for better convergence. In: Zitzler E, editor. Proceedings of the first international conference on evolutionary multi-criterion optimization 2001. Zurich, Germany, 2001. Lecture Notes in Computer Science no. 1993. London: Springer; 2001. p. 67–81.
[53] Chipperfield A, Fleming P, Polheim H, Fonseca C. Genetic algorithm toolbox user's guide. Research report 512, University of Sheffield, UK, 1994.
[54] Billings SA, Zheng GL. Radial basis function network configuration using genetic algorithms. Neural Networks 1995;8(6):877–86.
[55] Weile DS, Michielsen E. Integer coded Pareto genetic algorithm design of constrained antenna arrays. Electron Lett 1996;32(19):1744–5.
[56] Tao H, Liao G, Wang L. Integer coded genetic algorithm design of staggered sampling MTI. In: Proceedings of the IEEE international conference on neural networks and signal processing, Nanging, China, 2003, vol. 1. p. 558–62.
[57] Damousis IG, Bakirtzis AG, Dokopoulos PS. A solution to the unit-commitment problem using integer-coded genetic algorithm. IEEE Trans Power Syst 2004;19(2):1165–72.
[58] Li F, Pilgrim JD, Dabeedin C, Chebbo A, Aggarwal RK. Genetic algorithms for optimal reactive power compensation on the National Grid System. IEEE Trans Power Syst 2005;20(1):493–6.
[59] Mulenbein H, Schlierkamp-Voosen. Predictive models for the breeder genetic algorithm. Evolutionary Comput 1993;1(1):25–49.